



Universidade de Brasília

Instituto de Ciências Exatas
Departamento de Ciência da Computação

Técnicas de programação dinâmica para programação competitiva

Enzo Y. Niho

Monografia apresentada como requisito parcial
para conclusão do Bacharelado em Ciência da Computação

Orientador
Prof. Dr. Edson Alves da Costa Júnior

Brasília
2025



Universidade de Brasília

Instituto de Ciências Exatas
Departamento de Ciência da Computação

Técnicas de programação dinâmica para programação competitiva

Enzo Y. Niho

Monografia apresentada como requisito parcial
para conclusão do Bacharelado em Ciência da Computação

Prof. Dr. Edson Alves da Costa Júnior (Orientador)
FGA/UnB

Prof. Dr. Vinícius Ruela Pereira Borges Prof. Dr. Daniel Saad Nogueira Nunes
Universidade de Brasília Instituto Federal de Brasília

Prof. Dr. Marcelo Grandi Mandelli
Coordenador do Bacharelado em Ciência da Computação

Brasília, 20 de fevereiro de 2025

Dedicatória

Dedico este trabalho a toda a comunidade de programação competitiva, que ela continue crescendo no Brasil.

Agradecimentos

Agradeço primeiramente ao professor Edson, por ter aceitado me orientar e também me orientado de forma excelente, me ajudando a escrever este trabalho, sem ele o trabalho não teria o mesmo resultado. Meus agradecimentos ao professor Vinícius e Guilherme que também incentivaram muito os estudos no campus Darcy Ribeiro.

Gostaria de agradecer também aos meus amigos e colegas de equipe do Flamengo (Eduardo Quirino e Wilson Guimarães) e da Hello Kitty (Belphy e Bruno Vargas), que me acompanharam nas competições da Maratona, foi um prazer competir ao lado de vocês. Gostaria de agradecer também a todos participantes do grupo de estudos UnBalloon, que me introduziu ao mundo da maratona e me ensinou muito, em especial ao Alberto e ao Chico, que foram grandes mentores para os calouros.

Gostaria de agradecer também aos meus amigos e família pelo apoio durante toda essa jornada na graduação. Meus pais sempre foram prestativos e prezaram bastante pela dedicação aos estudos. Agradeço também a minha namorada, Rebeca Peniche Botelho de Araújo.

O presente trabalho foi realizado com apoio da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES), por meio do Acesso ao Portal de Periódicos.

Resumo

Este trabalho tem como objetivo ensinar a técnica de programação bastante conhecida, principalmente no mundo das competições, chamada de programação dinâmica. O objetivo é apresentar algumas técnicas introdutórias e depois apresentar algumas técnicas mais avançadas. A motivação para a realização deste trabalho veio pela falta de conteúdo em português sobre assuntos envolvendo Maratona de Programação. Mesmo em inglês, o conteúdo é bem escasso, sendo os principais recursos encontrados em blogs. Neste trabalho foram apresentadas seis técnicas clássicas e duas técnicas avançadas, totalizando a apresentação de oito problemas. Para cada técnica apresentada também foi mostrada a solução e um problema de um juiz online onde se utilizava a técnica. Ao final também foi sugerida uma lista com 17 problemas sobre as técnicas apresentadas para incentivar o leitor a resolvê-las. Para trabalhos futuros, é interessante avaliar a expansão de outras técnicas avançadas de programação dinâmica e também possivelmente trabalhar em outros tópicos de maratona, como Matemática, Estrutura de Dados e Processamento de Strings.

Palavras-chave: programação competitiva, Maratona de programação, programação dinâmica, técnicas de programação, algoritmo, ICPC, IOI

Abstract

This paper was written with the purpose of teaching dynamic programming to Portuguese speakers. It is intended to present basic types of dynamic programming and building up to more sophisticated techniques. As there is a few references in portuguese about competitive programming, this paper tries to extend the work of other people about this topic and strengthen competitive programming in Brazil. Eight techniques are explained in this paper, six of them are classic ones, while two of them are advenced. For each technique, there was one upsolving problem from an online judge. At last, there is a list of 17 problems from online judges to help the reader get used with this techniques. For future works, developing more sophisticated techniques or writing about other topics are very interesting for the Brazilian community.

Keywords: competitive programming, ICPC, IOI, dynamic programming

Sumário

1	Introdução	1
1.1	Trabalhos Correlatos	3
1.2	Objetivos	3
1.3	Estrutura do Trabalho	4
2	Referencial Teórico	5
2.1	Competições de programação	5
2.1.1	Conteúdo programático das competições	6
2.1.2	Juízes online	6
2.2	Paradigmas de solução de problemas	8
2.3	Programação Dinâmica	10
2.3.1	Aplicações de DP	11
2.3.2	Histórico	11
2.4	Conceitos elementares da Teoria de Grafos	12
3	DP's Clássicas	14
3.1	Problema da Mochila	14
3.1.1	Solução	15
3.1.2	Implementação	16
3.2	Problema do Troco	16
3.2.1	Solução	17
3.2.2	Implementação	19
3.3	Problema do Caixeiro Viajante	19
3.3.1	Solução	20
3.3.2	Implementação	21
3.4	Problema da maior subsequência crescente (LIS)	21
3.4.1	Solução	22
3.4.2	Implementação	23
3.5	Problema da maior subsequência comum (LCS)	24

3.5.1	Solução	24
3.5.2	Implementação	25
3.6	Problema da soma máxima de sublista contígua	25
3.6.1	Solução	26
3.6.2	Implementação	26
4	DP's Modernas	32
4.1	SOS DP	32
4.1.1	Solução	33
4.1.2	Implementação	34
4.2	Digit DP	34
4.2.1	Solução	34
4.2.2	Implementação	35
5	Aplicações de programação dinâmica na solução de problemas	38
5.1	AtCoder Educational DP Contest – Knapsack 2	38
5.1.1	Solução	39
5.1.2	Implementações	39
5.2	AtCoder Educational DP Contest – Frog 2	39
5.2.1	Solução	40
5.2.2	Implementações	40
5.3	CSES DP – Towers	40
5.3.1	Solução	41
5.3.2	Implementação	41
5.4	Educational Codeforces Round 123 – Increasing Subarray Sums	42
5.4.1	Solução	42
5.4.2	Implementação	43
5.5	SPOJ – MC	43
5.5.1	Solução	43
5.5.2	Implementação	44
5.6	AtCoder Begginer Contest 180 – Traveling Salesman among Aerial Cities	44
5.6.1	Solução	44
5.6.2	Implementação	45
5.7	AtCoder Educational DP Contest – Digit Sum	45
5.7.1	Solução	45
5.7.2	Implementação	46
5.8	CSES Additional Problems – Bit Problem	46
5.8.1	Solução	46

5.8.2	Implementação	47
5.9	Lista de problemas	47
6	Considerações Finais	59
6.1	Trabalhos Futuros	59
	Referências	61

Lista de Figuras

2.1	Problema “A idade de Dona Mônica” da OBI 2019 primeira fase nível sênior	7
2.2	Exemplo de grafo com vértices identificados de 1 a 5 e as seguintes arestas (1, 2), (2, 3), (3, 1), (2, 4), (5, 4)	13
3.1	Exemplo da árvore de recursão para o problema da mochila	15
3.2	Exemplo da árvore de recursão para o problema do troco	18

Lista de Tabelas

2.1	Quantidade de problemas por conteúdo da fase subregional da Maratona de Programação SBC no período de 2014 a 2023.	8
2.2	Alguns dos principais juízes online, com periodicidade de suas competições e tamanho aproximado do banco de problemas	9
2.3	Vereditos possíveis ao submeter um problema na Maratona de Programação SBC 2024	9
3.1	Exemplo de matriz de custo de distância entre cidades do Brasil.	20

Capítulo 1

Introdução

A *International Collegiate Programming Contest* (ICPC) foi uma das primeiras competições de programação, criada em 1978 [1]. Depois, muitas empresas como Google e Facebook também criaram suas próprias competições, como a Google Code Jam e a Meta Hackercup [2]. Nos últimos dez anos também se tornaram bastante populares juizes online, os quais hospedam competições semelhantes que ocorrem semanalmente de forma remota, como o Codeforces¹ e o AtCoder².

Essas competições trazem diversos benefícios para os competidores como: resolução de problemas [3]; se adaptar com problemas desconhecidos; trabalho sob pressão; trabalho em equipe (em competições que são em time); aprendizado de algoritmos e técnicas de programação e criatividade, entre outros [4]. É comumente associado também as entrevistas de grandes empresas, visto que em entrevistas técnicas costumam aparecer problemas de nível fácil a intermediário de programação competitiva. Desta forma essa facilidade com entrevistas técnicas é um grande atrativo para quem está começando na faculdade. Também é possível conhecer muitas pessoas do Brasil e do mundo ao participar de eventos presenciais e online, onde essas relações costumam trazer vários benefícios.

A ICPC é a maior competição de programação universitária do mundo. Em 2024, foram mais de 300.000 estudantes competindo para conseguir uma vaga na fase regional da ICPC [5]. Na fase regional, foram 73.083 estudantes com 12.258 técnicos em 3.424 universidades de 103 países. Os melhores classificados das oito regiões ICPC avançam para a final mundial, que em 2024 ocorreu em Astana, Cazaquistão. No Brasil, as universidades costumam realizar uma seletiva para escolher quais times irão representá-la na primeira fase da Maratona de Programação SBC [6], essas seletivas não são uma regra da competição, são feitas pois há uma limitação do número de vagas em uma sede. Os melhores colocados da primeira fase se classificam para a Final Brasileira da Maratona de Progra-

¹<https://codeforces.com>

²<https://atcoder.jp>

mação SBC. Novamente, os melhores colocados da Final Brasileira se classificam para a Final Regional da América Latina (uma das oito regiões ICPC). Por fim, os melhores times da América Latina se classificam para a ICPC World Finals.

O Brasil participa do ICPC desde 1996, sendo que a primeira participação brasileira foi da Unicamp na final mundial e resultou em uma menção honrosa³. Desde então o Brasil participou de todas as edições (apenas em 2000 não houve nenhum representante brasileiro na final mundial), sendo que a melhor colocação brasileira foi o décimo terceiro lugar conquistado pelo Instituto Tecnológico de Aeronáutica em 2006⁴. A UnB se classificou pela primeira vez para o mundial em 2018⁵, conquistando a 41^o colocação, e até o presente momento participou três vezes da final mundial.

Existem excelentes materiais em inglês para se preparar para o ICPC. Alguns dos livros famosos utilizados para treinamento são: “Competitive Programming 4: The Lower Bound of Programming Contests in the 2020s” [7] e “Guide to Competitive Programming: Learning and Improving Algorithms Through Contests” [8]. Alguns sites, como o Codeforces⁶, tem a funcionalidade de blogs, onde os competidores conseguem divulgar conhecimentos aos demais. Os principais juizes online são o Codeforces e o AtCoder, porém todas essas referências estão em inglês, o que pode caracterizar uma barreira linguística para quem quer começar a se preparar para essas competições.

Para tentar quebrar essa barreira, cada universidade mantém materiais próprios que são utilizados em cursos introdutórios para novos competidores. No Brasil, os grupos de estudo para programação competitiva são bem fortes e ajudam o iniciante a se ambientar nessas competições. Alguns exemplos são: UnBalloon (UnB), Monkeys (UFG), Brute (UDESC), GEMA (USP – São Carlos), Maratonusp (USP), IME++ (IME), entre outros. É comum que alguns membros destes grupos façam o trabalho de conclusão de curso sobre um tema da maratona: os alunos da USP que possuem um compilado de trabalhos desta natureza⁷. Nesta lista há um trabalho sobre programação dinâmica, porém com foco nas técnicas básicas de programação [9]. Algumas outras universidades também tem alunos com trabalhos sobre programação competitiva, como a UnB e a UFRJ, porém não há um registro local destes trabalhos.

Neste contexto, a motivação desse trabalho é incrementar e melhorar o material em português sobre programação competitiva. A escolha de programação dinâmica se deu pelo fato de ser um conteúdo bastante comum nessas competições e que muitas vezes constitui a base para a solução de diversos outros problemas.

³<https://icpc.global/community/results-1997>

⁴<https://icpc.global/community/results-2006>

⁵<https://icpc.global/community/results-2019>

⁶<https://codeforces.com/top>

⁷<https://www.ime.usp.br/~maratona/tccs.html>

1.1 Trabalhos Correlatos

O trabalho de Mattioli [10] faz um estudo do problema da moeda, comparando o algoritmo de programação dinâmica com o algoritmo guloso. Ele faz um estudo aprofundado neste problema, comparando a solução e eficiência por programação dinâmica ou por algoritmo guloso. O presente trabalho também abordará esse mesmo problema, porém será explorada apenas a solução de programação dinâmica.

O trabalho de Tommasini [9] também tem como tema programação dinâmica e é mais focado nos problemas clássicos que podem ser resolvidos por programação dinâmica. São abordados problemas como o da mochila, do troco e do caixeiro viajante. Também segue o mesmo estilo de apresentar questões de juízes online e mostrar a solução, além de uma lista de questões para praticar. O presente trabalho também aborda esses problemas, mas também apresenta técnicas avançadas para competidores mais experientes.

O trabalho de Santos [11] aborda um problema muito famoso nas competições, o problema do menor ancestral comum em uma árvore. Entre as soluções apresentadas, uma delas é baseada em programação dinâmica. No presente trabalho, também é abordado um problema de grafos bastante conhecido que é resolvido por programação dinâmica, o problema do caixeiro viajante.

1.2 Objetivos

O objetivo deste trabalho é apresentar a técnica de programação dinâmica para resolução de problemas de programação competitiva para competidores e estudantes de graduação de cursos de Computação. A meta é que este material consiga servir de preparação para qualquer estudante que esteja interessado em competir na Maratona de Programação SBC.

Os objetivos específicos são:

- apresentar técnicas que variam do básico ao avançado;
- apresentar problemas de juízes eletrônicos onde a programação dinâmica é um paradigma para resolução do problema; e,
- por fim, apresentar uma lista com sugestões de problemas para o leitor praticar o conteúdo visto no trabalho.

1.3 Estrutura do Trabalho

Este trabalho está dividido em quatro capítulos. O Capítulo 2 contextualiza o leitor sobre as competições de programação e suas regras. Também neste capítulo é fundamentada a parte teórica por trás da programação dinâmica, bem como o seu histórico e evolução. O Capítulo 3 apresenta problemas clássicos que são resolvidos através da programação dinâmica: o problema da mochila, o problema do troco, o problema do caixeiro viajante, o problema da maior subsequência crescente, o problema da maior subsequência comum e o problema da soma máxima de sublista contígua. Neste capítulo também há a solução com programação dinâmica e uma sugestão de implementação em C++. O Capítulo 4 segue o mesmo modelo do Capítulo 3, apresentando ao leitor problemas avançados: o problema da soma de subconjuntos e o problema da contagem de dígitos. Por fim, no Capítulo 5 são apresentados diversos problemas de juízes online, nos quais é necessário que seja aplicada a técnica de programação dinâmica para a solução. Apresenta-se o problema, os limites de entrada e saída e exemplos, por fim também apresenta a solução e uma sugestão de implementação em C++. Ao final do capítulo há uma lista com exercícios para o leitor praticar.

Capítulo 2

Referencial Teórico

Neste capítulo serão introduzidos termos importantes para este trabalho e também para ambientação no mundo da programação competitiva. Na Seção 2.1 será explicado como funcionam as principais competições de programação. Na Seção 2.2 serão apresentados os paradigmas de programação, técnicas que são utilizadas para resolver problemas. Na Seção 2.3 será dedicada à programação dinâmica, descrevendo desde a base teórica até os tipos de implementação e o histórico. Por fim, a Seção 2.4 apresenta conceitos elementares da teoria de grafos, pois uma das técnicas de programação dinâmica que será discutida é aplicada em grafos.

2.1 Competições de programação

Existem várias competições de programação em nível internacional (Meta Hacker Cup¹, *International Olympiads in Informatics* – IOI², *International Collegiate Programming Contest World Finals* – ICPC WF³) e nacional (Olimpiada Brasileira de Informática – OBI⁴, Maratona de Programação SBC⁵). As regras costumam ser parecidas e, em geral, se resumem em: os times competem entre si dentro de um tempo pré-determinado para resolver o maior número de problemas (o conjunto dos problemas é denominado, em geral, caderno de prova). Cada competição tem suas próprias regras [12], em algumas delas os times são compostos por três pessoas (ICPC e Maratona de Programação SBC) enquanto outras são individuais (Meta Hacker Cup e OBI). O tempo de duração da competição também varia, e algumas competições só atribuem pontos à equipe caso resolva o problema completamente, enquanto outras atribuem pontuação parcial.

¹<https://www.facebook.com/codingcompetitions/hacker-cup>

²<https://ioinformatics.org/>

³<https://icpc.global/>

⁴<https://olimpiada.ic.unicamp.br/>

⁵<https://maratona.sbc.org.br/>

As competições mais famosas, em nível universitário, são a ICPC WF e a Maratona de Programação SBC, que corresponde a fase classificatória no Brasil. A Maratona de Programação SBC possui duas fases, uma fase regional⁶ e uma fase nacional, e uma equipe precisa estar no topo do placar em ambas fases para se classificar para a final latino-americana, onde os melhores times da América Latina ganham o direito de competir na ICPC WF [13].

Este trabalho tem foco na Maratona de Programação SBC, mas o conteúdo de programação dinâmica é igualmente aplicável para qualquer competição descrita no início desta seção.

2.1.1 Conteúdo programático das competições

Os cadernos de provas são compostos, geralmente, de 10 a 15 problemas. Cada problema tem um título, um texto contextualizando o problema, os limites e restrições de entrada e saída, e exemplos de entrada e saída [14]. A Figura 2.1 exemplifica um problema do caderno de prova da OBI 2019.

Na Maratona de Programação SBC, os principais conteúdos que são necessários para a solução dos problemas são: Programação Dinâmica, Algoritmo Guloso, Teoria dos Números, Geometria Computacional, Teoria de Grafos, Processamento de Strings, Estrutura de Dados, Algoritmo de Força Bruta, Divisão e Conquista (D&C), Ad-Hoc⁷, Probabilidade, Álgebra, Combinatória e Teoria de Jogos. A Tabela 2.1 apresenta a relação aproximada de conteúdo por problemas (sendo aqueles problemas multidisciplinares contando em ambos conteúdos) da primeira fase da Maratona de Programação SBC no período 2014 a 2023, extraídos e computados pelo autor⁸. Nesses dez anos, foram 131 problemas com 46 deles sendo problemas multidisciplinares.

2.1.2 Juízes online

Além das competições citadas na Seção 2.1, existem vários juízes online que promovem suas próprias competições. Um juiz online engloba um juiz eletrônico, responsável por receber o código do usuário, compilar, testar com os casos explícitos e ocultos e dar o veredito final; e funcionalidades relativas à gerência de usuário e um banco de problemas para treino [2]. São ferramentas úteis para quem deseja competir ou treinar programação competitiva. A Tabela 2.2 lista alguns dos principais juízes online, com periodicidade

⁶Na nomenclatura original do ICPC, a fase regional do Brasil, onde os times se enfrentam em nível estadual, corresponde a etapa *subregional*. A etapa *regional* do ICPC equivale à fase nacional.

⁷Problemas que não tem uma categoria específica, requer um algoritmo muito específico

⁸Site para obter os cadernos de prova, onde foi feita a análise <https://maratona.sbc.org.br/hist/index.html>

Figura 2.1: Problema “A idade de Dona Mônica” da OBI 2019 primeira fase nível sênior

A idade de Dona Mônica

Nome do arquivo: “idade.x”, onde x deve ser c, cpp, pas, java, js, py2 ou py3

Dona Mônica é mãe de três filhos que têm idades diferentes. Ela notou que, neste ano, a soma das idades dos seus três filhos é igual à idade dela. Neste problema, dada a idade de dona Mônica e as idades de dois dos filhos, seu programa deve computar e imprimir a idade do filho mais velho.

Por exemplo, se sabemos que dona Mônica tem 52 anos e as idades conhecidas de dois dos filhos são 14 e 18 anos, então a idade do outro filho, que não era conhecida, tem que ser 20 anos, pois a soma das três idades tem que ser 52. Portanto, a idade do filho mais velho é 20. Em mais um exemplo, se dona Mônica tem 47 anos e as idades de dois dos filhos são 21 e 9 anos, então o outro filho tem que ter 17 anos e, portanto, a idade do filho mais velho é 21.

Entrada

A primeira linha da entrada contém um inteiro M representando a idade de dona Mônica. A segunda linha da entrada contém um inteiro A representando a idade de um dos filhos. A terceira linha da entrada contém um inteiro B representando a idade de outro filho.

Saída

Seu programa deve imprimir uma linha, contendo um número inteiro, representando a idade do filho mais velho de dona Mônica.

Restrições

- $40 \leq M \leq 110$
- $1 \leq A < M$
- $1 \leq B < M$
- $A \neq B$

Exemplos

Exemplo de entrada 1 52 14 18	Exemplo de saída 1 20
Exemplo de entrada 2 47 21 9	Exemplo de saída 2 21

Fonte: Site oficial da OBI [15].

Tabela 2.1: Quantidade de problemas por conteúdo da fase subregional da Maratona de Programação SBC no período de 2014 a 2023.

Conteúdo	Quantidade de problemas
Programação Dinâmica	19
Algoritmo Guloso	17
Teoria dos Números	14
Geometria Computacional	13
Teoria de Grafos	31
Processamento de Strings	7
Estrutura de Dados	25
Algoritmo de Força Bruta	16
Combinatória	8
Álgebra	5
Ad-Hoc	27
D&C	2
Probabilidade	4
Teoria de Jogos	2

Fonte: o Autor.

de suas competições e tamanho aproximado do banco de problemas, onde o símbolo ‘*’ significa “não se aplica”.

Após a submissão de um problema, o juiz eletrônico testa o código com vários casos escondidos e mostra um veredito final. A Tabela 2.3 mostra os principais vereditos que um competidor recebeu ao participar da primeira fase da Maratona de Programação SBC 2024⁹.

2.2 Paradigmas de solução de problemas

Dentre os conteúdos abordados nas competições (ver Seção 2.1), existem alguns paradigmas utilizados na solução de problemas. Os principais são: busca completa, divisão e conquista, estratégia gulosa e programação dinâmica [7].

A busca completa, como o próprio nome diz, usa a estratégia de buscar todo o espaço de solução em busca da correta [8]. Geralmente não possui uma implementação complexa

⁹https://maratona.sbc.org.br/primfase24/contest-pf2024/info_maratona.pdf

Tabela 2.2: Alguns dos principais juizes online, com periodicidade de suas competições e tamanho aproximado do banco de problemas

Juiz Online	Periodicidade	Banco de problemas
Codeforces	3	10.000
AtCoder	2	7.500
CodeChef	1	100.000
CSES	*	300
SPOJ	*	42.488
NEPS	*	1.500

Fonte: o Autor.

Tabela 2.3: Vereditos possíveis ao submeter um problema na Maratona de Programação SBC 2024

Resultado	Sigla	Significado
<i>YES</i>	AC	<i>Accepted</i>
<i>NO</i>	WA	<i>Wrong Answer</i>
<i>NO</i>	TLE	<i>Time Limit Exceeded</i>
<i>NO</i>	CE	<i>Compilation Error</i>
<i>NO</i>	RTE	<i>Run Time Error</i>

Fonte: Maratona de Programação SBC 2024 [16].

e é preferível em casos que o espaço de solução é relativamente pequeno, justamente pela facilidade de implementação. Porém, caso o espaço seja relativamente grande, se torna inviável uma busca completa pela resposta correta, o que pode acabar resultando em um veredito TLE. Por exemplo, para encontrar uma palavra em um dicionário, pode-se realizar uma busca completa, olhando um a um todos os termos, para localizar a palavra desejada.

A estratégia gulosa consiste em dividir o problema original em subproblemas e, a cada passo do algoritmo, fazer uma escolha local de uma maneira que ela resultará na melhor escolha global, ou seja, a escolha local não depende de outros subproblemas, nem das escolhas locais anteriores, para determinar a melhor escolha [17]. Também não costuma ter uma implementação complexa, mas geralmente é difícil provar que a estratégia funciona. É comum os competidores terem uma intuição de que a abordagem gulosa funciona e tentam resolver a questão por esse método. Por exemplo, verificar se um grafo é bipartido

pode ser resolvido por meio de uma estratégia gulosa.

A divisão e conquista consiste em dividir o problema em duas ou mais partes até que o problema se torne pequeno o suficiente, ao ponto em que possa ser resolvido trivialmente. Após isso, é feita uma junção das soluções dos subproblemas já resolvidos, gerando a solução final [8]. Este paradigma costuma levar a implementações mais elaboradas, porém ela pode ser aplicada em diversos problemas, resultado em algoritmos com complexidade assintótica relativamente baixa. Por exemplo, o algoritmo *mergesort* [17] é baseado no paradigma de divisão e conquista.

A programação dinâmica será abordada na Seção 2.3.

2.3 Programação Dinâmica

A programação dinâmica é um paradigma de solução de problemas que consiste em dividir o problema em subproblemas e obter a solução a partir de uma combinação das soluções destes subproblemas. Diferencia-se dos outros paradigmas pois, para poder ser utilizado, o problema deve ter duas propriedades. A primeira propriedade é o princípio de otimização de Bellman (Definição 1), que diz que em uma solução ótima existem soluções ótimas de seus subproblemas. A segunda propriedade é ter a superposição de subproblemas.

Definição 1 *O princípio de otimização de Bellman, também conhecido como técnica de otimização recursiva, afirma que uma estratégia ótima precisa ser uma que, independente de como se chega a um estado, todas as decisões após esse estado devem ser ótimas [18].*

Em outros termos, para que um problema possa ser resolvido com programação dinâmica, ele precisa ter subestrutura ótima e superposição de subproblemas. A subestrutura ótima é implicada pelo princípio de otimização de Bellman. A superposição de subproblemas ocorre quando diversos estados precisam visitar um mesmo estado, podendo gerar uma complexidade exponencial. Para visitar menos estados, é possível utilizar tabelas para memorizar os estados que foram visitados previamente. Um algoritmo de programação, aplicado em um problema com subestrutura ótima, mas sem superposição de subproblemas, equivale a um algoritmo recursivo de busca completa. Será utilizado o termo DP (do original inglês *dynamic programming*) para se referir à programação dinâmica. Na descrição de alguns algoritmos de DP será utilizada a notação P_i para indicar um prefixo de tamanho i de P .

Existem alguns modos de implementar técnicas baseadas em programação dinâmica. A implementação *top-down* geralmente é feita de forma recursiva e executa os casos bases por último. A implementação *bottom-up* geralmente é feita de forma iterativa e constrói a resposta do caso base até a resposta desejada. Existem outras técnicas, como *Push DP*,

que é quando o estado atual computa a resposta dos próximos estados. Outra técnica é *Pull DP*, que é o inverso da anterior, então o estado atual é computado pela resposta dos estados anteriores [19].

2.3.1 Aplicações de DP

Existem muitos problemas que são facilmente resolvidos com DP. Outros também utilizam DP como um passo para a obtenção da solução. No contexto de programação competitiva, é uma ferramenta de extrema importância para os competidores, pois muitos problemas podem ser abordados por meio dessa técnica. Fora do ambiente competitivo, também é interessante ter conhecimento dessa técnica, pois muitas vezes pode-se ter um ganho de tempo de processamento ao se utilizar programação dinâmica para acelerar alguns processos no ambiente em que se trabalha.

Programação dinâmica é um paradigma extenso, sendo utilizado não apenas nas competições de programação, mas na área de ciência da computação como um todo. Também é aplicada nas mais diversas áreas, desde engenharia a economia [20], mas também em inteligência artificial e bioinformática [21].

2.3.2 Histórico

A invenção da programação dinâmica é creditada ao matemático e cientista da computação Richard Bellman [22]. Ele precisava criar uma técnica para resolver problemas de processos de decisão em vários estágios. A técnica para resolver esses problemas acabou sendo nominada por ele de programação dinâmica. Nesse processo Bellman também formalizou o princípio de otimização (ver Definição 1).

O termo *dynamic programming* foi cunhado por Richard Bellman, que conta a origem do nome em sua autobiografia “*Eye of the Hurricane*” [23, pg. 159]. Em resumo, ele estava trabalhando na RAND Corporation e durante a década de 1950, existia um secretário de defesa que, segundo Bellman, era aterrorizado pela palavra pesquisa. Na época, a empresa que ele trabalhava prestava serviço para a força aérea e a força aérea tinha o secretário de defesa como seu chefe. Então, para proteger a empresa e suas pesquisas, ele teve que pensar em um nome que não soasse mal aos ouvidos do secretário de defesa.

Ele pensou em usar a palavra “planejamento”, mas não era um bom termo por diversos motivos, então acabou escolhendo a palavra “programação”. Além disso, era necessário que a solução fosse relacionada ao dinamismo, multiestágio, que variasse de acordo com o tempo. Então foi decidido o termo “dinâmica” para completar a nomenclatura dessa técnica. De acordo com Bellman, é impossível utilizar a palavra “dinâmica” com uma conotação pejorativa. Então a escolha foi a expressão “programação dinâmica”, a qual

nenhum deputado iria se opor. Com isso ele utilizou esta designação como um escudo para suas atividades na empresa.

O principal marco histórico ocorreu na década de 1950, quando Bellman criou o termo e o popularizou na comunidade. A técnica foi bastante difundida, porém não houve registros das primeiras soluções de problemas famosos a serem resolvidos por programação dinâmica, como o problema do troco e do caixeiro viajante. Uma das primeiras menções registradas do problema da mochila foi em 1897, neste artigo de Mathews [24]. Como a invenção da programação dinâmica foi desenvolvida apenas meio século depois e não houve registro, não se sabe ao certo quando foi criado, mas já era um termo conhecido em 1975, como registrado por Salkin [25]. Outra solução que ficou bastante conhecida e registrada foi a solução de J. Kadane [26] para o problema de soma máxima de sublista contígua.

2.4 Conceitos elementares da Teoria de Grafos

Para a Definição 8, é necessário o conhecimento em termos básicos de grafos. O indicado é que o leitor tenha esse conhecimento prévio, mas caso queira se aprofundar, pode consultar o Cormen [17].

Na Definição 2 é definido o elemento mais básico da teoria de grafos, que é o próprio grafo composto por vértices e arestas. Na Figura 2.2 temos um exemplo de grafo com 5 vértices e 5 arestas.

Definição 2 *Seja V um conjunto e E uma relação de V em V , isto é, $E \subset V \times V$. Dizemos que $G = G(V, E)$ é um grafo. Os elementos $v \in V$ são denominados vértices, e se $(u, v) \in E$, dizemos que existe uma aresta partindo do vértice u e chegando no vértice v .*

Na Definição 3 define-se o conceito de caminho, que é uma sequência de vértices no grafo que são conectados por arestas. Na Figura 2.2, existe um caminho entre 1 e 4, utilizando as seguintes arestas: $(1, 2)$ e $(2, 4)$.

Definição 3 *Seja $G(V, E)$ um grafo, onde V é o conjunto de vértices do grafo e E o conjunto das arestas de G . Dizemos que existe um caminho de $u \in V$ até $v \in V$ se existe uma sequência de vértices distintos $\{a_i\}$ de tamanho N cujo primeiro vértice $a_1 = u$, o último vértice $a_N = v$ e para todo par adjacente na lista $(a_i, a_{i+1}) \in E$.*

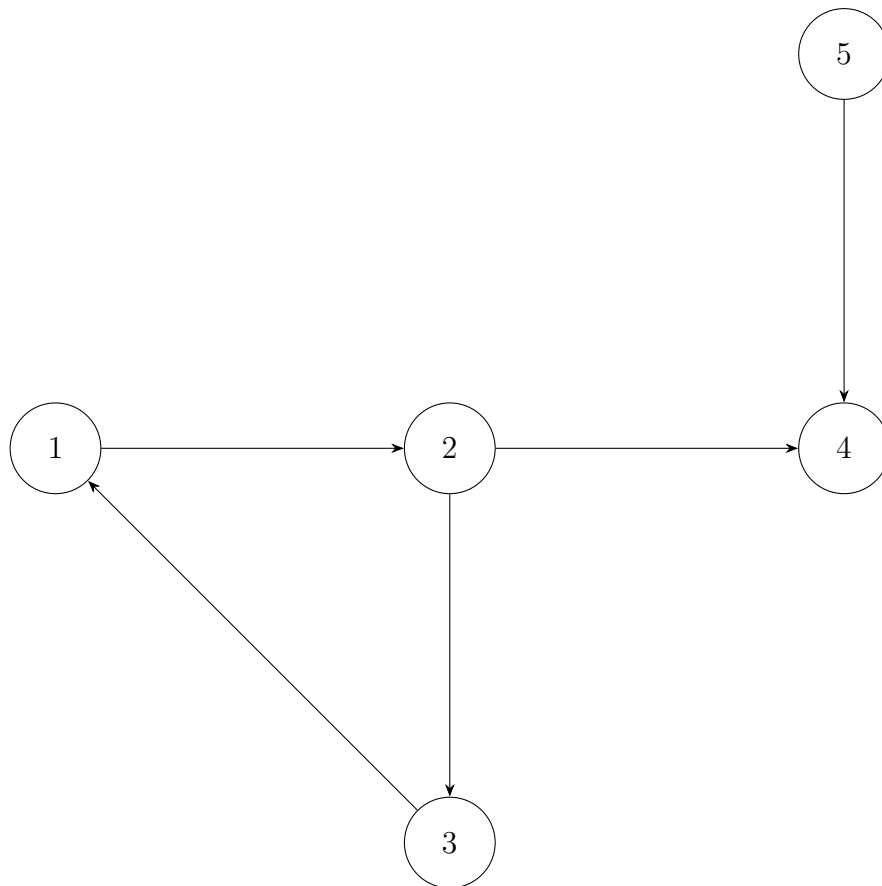
Na Definição 4 é definido o tamanho de um caminho, que é basicamente a quantidade de arestas no caminho. Na Figura 2.2 o tamanho do caminho dos vértices 1 a 4 é dado pela quantidade de arestas, logo o tamanho é dois.

Definição 4 Seja $G(V, E)$ um grafo e $\{a_i\}$ os vértices de um caminho nesse grafo. Dizemos que o tamanho do caminho é o número de arestas no caminho, logo o tamanho do caminho é a quantidade de vértices de $\{a_i\}$ menos um.

Na Definição 5 é definido um caminho especial chamado ciclo, onde o primeiro e último vértice do caminho são iguais. Na Figura 2.2 temos um ciclo definido pelas seguintes arestas: $(1, 2)$, $(2, 3)$ e $(3, 1)$.

Definição 5 Seja $G(V, E)$ um grafo, $\{a_i\}$ os vértices que representam um caminho nesse grafo e N a quantidade de vértices no caminho. Dizemos que $\{a_i\}$ é um ciclo se o primeiro e o último vértices são iguais, ou seja, se $a_1 = a_N$.

Figura 2.2: Exemplo de grafo com vértices identificados de 1 a 5 e as seguintes arestas $(1, 2)$, $(2, 3)$, $(3, 1)$, $(2, 4)$, $(5, 4)$



Fonte: o Autor.

Capítulo 3

DP's Clássicas

Neste capítulo serão introduzidas algumas soluções de problemas baseadas em programação dinâmica que são ensinadas nos cursos introdutórios de programação competitiva. São técnicas que foram desenvolvidas há muito tempo e são amplamente utilizadas em várias áreas. É importante frisar que mesmo sendo clássicas, ainda existem vários problemas com variações dessas técnicas presentes nas competições de programação.

3.1 Problema da Mochila

Definição 6 *Sejam W um inteiro positivo e $\{a_k\}$ uma sequência de N pares $a_i = (w_i, v_i)$, com $w_i \geq 0$ para todo $i = 1, 2, \dots, N$. O problema da mochila binária (knapsack, em inglês) consiste em determinar uma subsequência de índices $1 \leq i_1 < i_2 < \dots < i_r \leq N$ tal que*

$$\sum_{j=1}^r w_{i_j} \leq W \quad (3.1)$$

e que maximize a soma

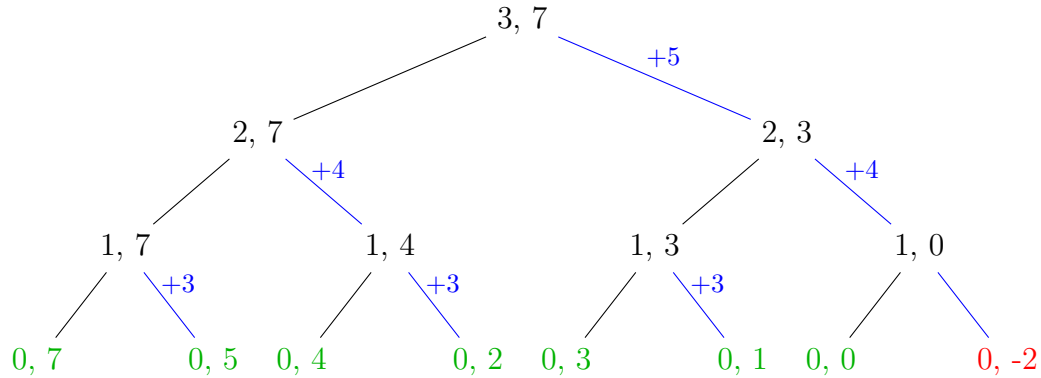
$$S = \sum_{j=1}^r v_{i_j}. \quad (3.2)$$

Os elementos do problema apresentado na Definição 6 podem ser interpretados como uma mochila que pode carregar W unidades de massa e uma lista de N itens, cada um possuindo um valor v_i e uma massa w_i . O problema consiste em escolher alguns desses itens tais que a soma das massas dos itens escolhidos não ultrapasse W e que a soma dos valores desses itens seja a maior possível.

Por exemplo, considere uma mochila com capacidade $W = 7$ e uma lista a de 3 itens com os respectivos valores e pesos iguais a $(3, 2), (4, 3), (5, 4)$. Se forem escolhidos os itens 2, 3, isso significa que a massa a ser carregada na mochila será de 7 e o valor total dos

itens será igual a 9. A Figura 3.1 ilustra uma árvore de recursão do exemplo, onde as arestas pretas são a transição onde não é escolhido o elemento e as arestas em azul são escolhidas os elementos. Cada vértice representa um estado, composter por um par de índice e peso da mochila.

Figura 3.1: Exemplo da árvore de recursão para o problema da mochila



Fonte: o Autor.

3.1.1 Solução

O problema da mochila pode ser resolvido por meio de uma função $dp(i, c)$ que calcule a melhor resposta para os elementos $\{a_1, \dots, a_i\}$ e uma mochila com capacidade c . O valor de $dp(i, c)$ pode ser computado a partir da solução do problema que considera apenas os $i - 1$ primeiros itens. Há duas possibilidades: não escolher o i -ésimo item ou tentar pegar esse item (caso tenha espaço na mochila), nesse caso tendo que calcular a melhor resposta no prefixo $i - 1$ e $c - p$ em que p é o peso do i -ésimo item. Há um único caso base, que ocorre quando não há mais itens a serem considerados.

Formalmente, seja $dp(i, c)$ o valor máximo que pode ser obtido a partir i primeiros itens da sequência ($1 \leq i \leq N$), com uma mochila de capacidade $c \geq 0$. Há dois casos base:

1. não há nenhum item a ser escolhido. Neste caso, o valor máximo é zero, independentemente da capacidade da mochila; e
2. não há espaço na mochila. Neste caso, o valor máximo também é zero, pois não é possível colocar nenhum item na mochila.

Os casos-base correspondem aos casos em que $i = 0$ e $w = 0$ apresentados na Equação 3.3.

Para os demais subproblemas, há duas transições possíveis:

1. ignorar o i -ésimo item; e
2. caso $w_i \leq c$, escolher o i -ésimo item.

Como a solução do problema deve maximizar a soma dos valores, então a resposta para o subproblema $\text{dp}(i, c)$ será a melhor dentre as duas transições, conforme ilustra a Equação 3.3.

$$\text{dp}(i, w) = \begin{cases} 0, & \text{se } i = 0 \text{ ou } w = 0 \\ \max(\text{dp}(i-1, w), \text{dp}(i-1, w - w_i) + v_i), & \text{se } w_i \leq w \\ \text{dp}(i-1, w), & \text{caso contrário} \end{cases} \quad (3.3)$$

A solução do problema será dada por $\text{dp}(N, W)$. O número de estados é $\Theta(N \times W)$ e cada transição é feita em $\Theta(1)$, de modo que esta solução tem complexidade de tempo e memória de $\Theta(N \times W)$.

3.1.2 Implementação

O Código 3.1 apresenta uma possível implementação da solução em C++.

3.2 Problema do Troco

Definição 7 *Seja C um inteiro positivo e $\{a_k\}$ um conjunto de N inteiros positivos. O problema do troco (Coin Change Problem, em inglês) consiste em escolher $\{u_r\}$ inteiros não negativos tal que*

$$C = \sum_{j=1}^r a_j \times u_j \quad (3.4)$$

A Definição 7 apresenta um problema que pode ser visualizado da seguinte forma. Suponha que em um mercado o caixa tenha uma quantidade ilimitada de cédulas de cada valor e precisa dar um troco de C reais para o cliente. O problema consiste em informar se o caixa consegue dar o troco para o cliente ou não.

Por exemplo, suponha que o caixa precisa devolver um troco $C = 7$ e possui as seguintes cédulas cujos valores são $\{2, 3\}$, com quantidades arbitrárias de cada cédula, ele é capaz de devolver o troco caso escolha as notas $\{2, 3\}$ com 2 notas de valor 2 e 1 nota de valor 3. A Figura 3.2 mostra a árvore de recursão para o exemplo citado, em preto é a transição utilizando a moeda de valor 2 e em azul é a transição utilizando a moeda de valor 3. O valor de cada nó representa o estado, logo para esse exemplo pode

Código 3.1: Implementação da solução do problema da Mochila em C++

```
1 constexpr int oo = 1e9; // constante infinito
2 constexpr int MAXN = 1e4; // valor máximo que a variável N pode ter
3 constexpr int MAXW = 1e3; // valor máximo que a variável W pode ter
4 int N, W; // N = #itens, W = capacidade da mochila
5 vector<int> w(MAXN+2), v(MAXN+2); // vetores para guardar o peso e o valor de cada item
6 vector<vector<int>> dp(MAXN+2, vector<int>(MAXW+2, -1)); // tabela para memoizar o resultado
7
8 int RecursiveKnapsack(int i, int c) { // resposta em RecursiveKnapsack(N, W);
9
10     if(i == 0) { // caso base em que acabaram os itens a serem analisados
11         return 0;
12     }
13
14     auto& ans = dp[i][c]; // memoizando
15     if(~ans) return ans;
16
17     int caso1, caso2 = -oo;
18     caso1 = RecursiveKnapsack(i-1, c); // caso em que não se pega o item atual
19
20     if(c - w[i-1] >= 0) // caso em que se pega o item atual se tem espaço na mochila
21         caso2 = RecursiveKnapsack(i-1, c - w[i-1]) + v[i-1];
22
23     return ans = max(caso1, caso2); // retorna o máximo entre os dois
24 }
25
26 int IterativeKnapsack() {
27
28     for(int i = 0; i <= W; i++) // construindo o caso base
29         dp[0][i] = 0;
30
31     for(int i = 1; i <= N; i++) {
32         for(int c = 0; c <= W; c++) {
33             int caso1, caso2 = -oo;
34             caso1 = dp[i-1][c]; // opção de não pegar o item atual
35             if(c - w[i-1] >= 0) // opção de pegar o item atual;
36                 caso2 = dp[i-1][c - w[i-1]] + v[i-1];
37             dp[i][c] = max(caso1, caso2);
38         }
39     }
40     return dp[N][W];
41 }
```

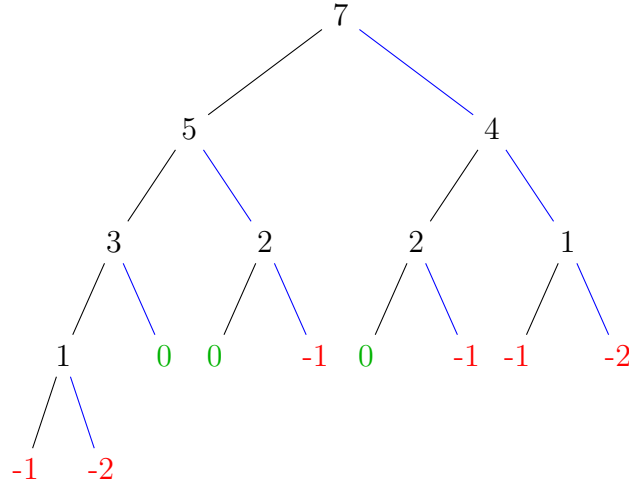
Fonte: o Autor

ser observado a super posição de alguns estados, como o estado de valor 2 e 1. Os estados em cor vermelha representam trocos inválidos e em cor verde representam o caso base, em que é possível devolver o troco.

3.2.1 Solução

Uma solução pode ser obtida por meio de uma função $dp(C, N)$ que determina se é possível dar um troco de valor C usando uma quantidade arbitrária de cédulas do conjunto $\{a_i\}$ de tamanho N . Uma maneira de computar esta função consiste em testar todas cédulas de valor a_i que não ultrapassem o valor C e verificar se é possível dar um troco $C - a_i$.

Figura 3.2: Exemplo da árvore de recursão para o problema do troco



Fonte: o Autor.

O caso base ocorre quando o troco é menor ou igual a zero: caso seja negativo, sabemos que é impossível dar o troco, caso seja zero, sabemos que é possível dar o troco.

Formalmente, seja $dp(c)$ uma função que indica se é possível ou não dar um troco de c usando uma quantidade arbitrária de cédulas do conjunto $\{a_i\}$ de tamanho N . Há dois casos bases:

1. o troco é negativo: não é possível dar um troco negativo; e
2. o troco é igual a zero: é possível dar um troco vazio.

Os casos-base são apresentados na Equação 3.5, nos casos em que i é não positivo.

Para os demais subproblemas, existem N transições possíveis: para cada cédula disponível, verifique se é possível usar uma delas para dar o troco.

Como a solução do problema verifica a existência de um troco, então é possível checar se existe pelo menos uma transição em que o troco é viável. A transição é dada na Equação 3.5, onde o índice é positivo.

$$dp(i) = \begin{cases} \text{false}, & \text{se } i < 0 \\ \text{true}, & \text{se } i = 0 \\ \bigvee_{j=1}^N dp(i - a_j), & \text{caso contrário} \end{cases} \quad (3.5)$$

onde $\vee$ é o “ou” lógico.

A solução do problema é dada por $dp(C)$. O número de estados é $\Theta(C)$ e as transições são feitas em $\Theta(N)$, de modo que a solução tem complexidade $\Theta(C \times N)$. Como é preciso memorizar a resposta para cada estado, a complexidade de memória é $\Theta(C)$.

3.2.2 Implementação

O Código 3.2 apresenta uma possível implementação da solução em C++.

Código 3.2: Implementação da solução do problema do troco em C++

```
1 constexpr int oo = 1e9; // valor infinito
2 constexpr int MAXN = 1e5; // valor maximo que a variavel N pode ter
3 int N, C; // quantidade de cedulas e o troco a ser dado
4 vector<int> a(MAXN+2); // vetor com os valores das cedulas
5 vector<int> dp;
6
7 int RecursiveCoinChange(int i) { // resposta vai estar em RecursiveCoinChange(C)
8
9     if(i == 0) return 1; // é possível dar o troco
10
11     auto& ans = dp[i];
12     if(~ans) return ans;
13     ans = 0; // como não verificamos nada ainda, é impossível dar o troco
14
15     for(int j = 1; j <= N; j++) {
16         if(troco - cedulas[j] >= 0) {
17             ans = ans || RecursiveCoinChange(i - a[j]);
18         }
19     }
20     return ans;
21 }
22
23 int CoinChange() {
24
25     dp[0] = 1;
26     for(int i = 1; i <= C; i++)
27         dp[i] = 0;
28
29     for(int i = 1; i <= C; i++) {
30         for(int j = 1; j <= N; j++) {
31             if(i - a[j] >= 0) {
32                 dp[i] = dp[i] || dp[i - a[j]];
33             }
34         }
35     }
36     return dp[C];
37 }
```

Fonte: o Autor

3.3 Problema do Caixeiro Viajante

Definição 8 Seja $G(V, E)$ um grafo com N vértices $V = \{u_n\}$ e M arestas $E = \{v_m\}$, onde $v_m = (u_i, u_j, w_m)$ para algum par de índices (i, j) , $1 \leq i, j \leq N$, e w_m é o custo da m -ésima aresta, $1 \leq m \leq M$. O problema do caixeiro viajante (Traveling Salesman Problem - TSP, em inglês) consiste em identificar um ciclo $u_{i_1}, u_{i_2}, \dots, u_{i_{N+1}}$ tal que $u_{i_1} = u_{i_{N+1}}$ e que, para cada vértice u_i , existe pelo menos um índice j tal que $u_i = u_{i_j}$ e cujo custo (soma dos pesos das arestas) é mínimo, ou seja, identificar um ciclo que minimize a Equação

$$\sum_{j=1}^N C(u_{i_j}, u_{i_{j+1}}), \quad (3.6)$$

onde $C(u_{i_j}, u_{i_{j+1}})$ é o custo da aresta que conecta os vértices u_{i_j} e $u_{i_{j+1}}$.

A Definição 8 apresenta um problema que pode ser visualizado da seguinte forma. Suponha que há um caixeiro viajante em uma cidade S que deseja visitar todas as cidades do país e voltar para a cidade S de tal forma que a distância percorrida seja mínima. O problema consiste em escolher uma ordem de cidades que o caixeiro irá visitar para que a distância seja a mínima possível.

Tabela 3.1: Exemplo de matriz de custo de distância entre cidades do Brasil.

	Brasília	Belém	João Pessoa	Porto Alegre
Brasília	0	20	50	40
Belém	20	0	30	80
João Pessoa	50	30	0	90
Porto Alegre	40	80	90	0

Fonte: o Autor.

Por exemplo, suponha que o caixeiro comece em Brasília e queira visitar as seguintes cidades: Belém, João Pessoa e Porto Alegre. As distâncias entre uma cidade e outra são dadas pela matriz apresentada na Tabela 3.1. Uma possível resposta é (Brasília, João Pessoa, Porto Alegre, Belém, Brasília) com custo 240. Neste caso, como são poucas cidades, podemos testar todas as permutações de cidade e observar que a melhor resposta é (Brasília, Belém, João Pessoa, Porto Alegre, Brasília).

3.3.1 Solução

Uma solução pode ser obtida por meio da uma função $dp(i, T)$ que determina a menor distância percorrida para partir de i até a cidade inicial tendo visitado todas as cidades do conjunto T . Para calcular essa função, para cada cidade j que não está no conjunto T é necessário verificar a resposta para esta cidade com o restante dos elementos de T mais a distância de j até i : a resposta será o menor valor entre todas as cidades que não estão em T . O caso base ocorre quando todas as cidades já foram visitadas, no qual o custo será a distância da cidade atual até a cidade inicial.

Formalmente, seja $dp(i, T)$ o menor custo para visitar todas as cidades que não estão em T começando em i e terminando na cidade inicial. O único caso base é quando o conjunto T contém todas as N cidades: então o custo é a distância da cidade i até S .

O caso base é apresentado na Equação 3.7 quando $T = V$.

Para os demais subproblemas, podemos ter N transições possíveis: para cada cidade que não foi visitada, verifique o menor custo de visitar todas as cidades que não estão no conjunto T mais o custo de viajar da cidade atual para a cidade que será visitada.

A transição é dada na Equação 3.7

$$\text{dp}(i, T) = \begin{cases} C(i, S), & \text{se } T = V \\ \min_{j \notin T} (\text{dp}(j, T \cup \{j\}) + C(i, j)), & \text{caso contrário.} \end{cases} \quad (3.7)$$

A solução do problema é dada por $\text{dp}(S, \emptyset)$. O número de estados é $\Theta(N \times 2^N)$ e a transição é $\Theta(N)$. Assim a complexidade de tempo é $\Theta(N^2 \times 2^N)$. Como é preciso memorizar cada estado a complexidade de memória é $\Theta(N \times 2^N)$.

3.3.2 Implementação

O Código 3.3 apresenta uma possível implementação da solução em C++.

3.4 Problema da maior subsequência crescente (LIS)

Definição 9 *Seja $\{a_k\}$ uma sequência de N inteiros. O problema da maior subsequência crescente (Longest Increasing Subsequence – LIS, em inglês) consiste em escolher a maior subsequência de índices $1 \leq i_1 < i_2 < \dots < i_r \leq N$ tal que*

$$a_{i_j} < a_{i_{j+1}}, \quad \forall 1 \leq j \leq r - 1 \quad (3.8)$$

A Definição 9 apresenta um problema que pode ser visualizado da seguinte forma. Suponha que há um jogo de N rodadas, a cada rodada o jogador escolhe adicionar a carta em sua pilha ou não, podendo adicionar a carta apenas se o valor da carta for maior que o valor da carta no topo da pilha. Dado que você sabe os valores das N cartas, o problema consiste em responder qual a quantidade máxima de cartas na pilha que o jogador pode ter se jogar de forma ótima.

Por exemplo, suponha que as cartas serão jogadas na seguinte ordem $\{1, 2, 1, 4, 7, 5, 10\}$. Uma possível sequência de cartas escolhidas é $\{2, 4, 7\}$. A resposta que obtém a maior quantidade de cartas na pilha seria $\{1, 2, 4, 5, 7, 10\}$.

3.4.1 Solução

Solução Quadrática

Um solução pode ser obtida por meio de uma função $dp(i)$ que calcula a maior subsequência crescente que termina em i . Para cada prefixo de tamanho j , com $j < i$, deve-se checar o maior valor da função de j se a_j é menor que a_i . O caso base é quando estamos na primeira posição, nesse caso o valor da função é 1.

Formalmente, seja $dp(i)$ a maior subsequência crescente que termina em i . O único caso base é quando o primeiro elemento é calculado: a única subsequência possível é formada apenas pelo elemento a_1 . O caso-base é apresentado na Equação 3.9 quando $i = 1$.

Para os demais subproblemas, existem N transições possíveis: verificar se, ao anexar o elemento a_i à maior subsequência crescente terminada em a_j , obtém-se uma subsequência maior do que as já obtidas para os prefixos anteriores. Caso não haja nenhum elemento, a maior sequência tem tamanho um, que é o próprio elemento. A transição é dada pela Equação 3.9

$$dp(i) = \begin{cases} 1, & \text{se } i = 1 \\ 1 + \max_{1 \leq j < i} (dp(j)) & \forall a_j < a_i, \text{ se } \exists a_j < a_i \\ 1, & \text{caso contrário.} \end{cases} \quad (3.9)$$

Para se obter o tamanho da maior subsequência crescente basta identificar a maior subsequência que termina em cada prefixo e guardar o tamanho da maior delas. Se a sequência tem N elementos e a complexidade para checar o máximo é $\Theta(N)$ então tem uma complexidade de tempo $\Theta(N^2)$. Como é preciso guardar o tamanho máximo para cada prefixo, a complexidade de memória é $\Theta(N)$.

É possível melhorar a complexidade desta solução ao utilizar estruturas de dados que consultam o maior elemento em um intervalo e que conseguem atualizar um elemento, como a Árvore de Segmentos [8]. Essa estrutura tem complexidade de tempo $\Theta(\log N)$ para consulta, melhorando a complexidade de tempo para $\Theta(N \times \log N)$ e essa estrutura precisa de $\Theta(N)$ de memória.

Solução $\Theta(N \log N)$

Para obter esta mesma complexidade de tempo sem utilizar estruturas de dados avançadas, deve-se modelar o problema de uma forma diferente. Uma solução pode ser obtida através do método de aproximações sucessivas [19]. Utiliza-se uma lista d , onde $d[i]$ é o último menor elemento de uma sequência de tamanho i . Inicializa-se a lista d , onde

$d[0] = -\infty$ e para os outros índices i , $d[i] = \infty$. Então processamos os elementos da esquerda para a direita, na tentativa de melhorar a resposta em d .

Para cada a_i , é verificado todos os índices j que satisfazem $d[j-1] < a_i < d[j]$. Dada essa condição, percebe-se que apenas um valor de d é atualizado. A lista d é monotônica, pois se é retirado o último elemento da LIS de tamanho i , obtém-se uma LIS de tamanho $i-1$, na qual o último número é ainda menor.

Como a lista d é monotônica e só é necessário atualizar um valor, pode-se utilizar busca binária para encontrar o valor a ser atualizado. Deve-se buscar pelo primeiro elemento que seja estritamente maior que a_i e atualizar esse valor por a_i . Por fim, para obter a resposta, basta verificar o maior índice i em d que seja menor que ∞ .

A recorrência é apresentada na Equação 3.10,

$$d_i = \begin{cases} \epsilon, & \text{se } i = 0, \\ f_i, & \text{se } i \leq N, \\ \emptyset, & \text{caso contrário,} \end{cases} \quad (3.10)$$

onde $\epsilon = [-\infty, \infty, \dots, \infty]$, $|\epsilon| = N$ e $f_i = F(d_{i-1}, a_i)$ é uma nova lista tal que $f_i = d_{i-1}$, exceto em, no máximo um índice $j \in [1, N]$, o qual satisfaz à condição apresentada na Equação 3.11:

$$f_i[j] = a_i \text{ se } d_{i-1}[j-1] < a_i < d_{i-1}[j]. \quad (3.11)$$

Em outros termos, a Equação 3.10 mostra que, em $i = 0$, o caso base é uma lista ϵ de N inteiros, o primeiro elemento sendo $-\infty$ e os demais são iguais a ∞ . Para os outros casos, a transição consiste em tentar atualizar a lista anterior com o i -ésimo elemento a_i .

A solução do problema é o maior índice i em d_N que seja menor que ∞ . Como é necessário processar todos os N elementos da lista e para cada um deles realizar uma busca binária, a complexidade de tempo é $\Theta(N \times \log N)$. Como temos N listas de tamanho N , a complexidade de memória seria $\Theta(N^2)$, porém como é feito apenas uma atualização em cada iteração, é possível manter a mesma lista e atualizá-la ao processar cada elemento, dessa forma a complexidade de memória reduz para $\Theta(N)$.

3.4.2 Implementação

Uma possível implementação para solução quadrática é apresentada no Código 3.4. O Código 3.5 traz uma implementação da solução $\Theta(N \log N)$ [27].

3.5 Problema da maior subsequência comum (LCS)

Definição 10 *Seja S uma string de tamanho N e T uma string de tamanho M . O problema da maior subsequência comum (Longest Common Subsequence – LCS, em inglês) consiste em determinar duas subsequências de índices $1 \leq a_1 < a_2 < \dots < a_r \leq N$ e $1 \leq b_1 < b_2 < \dots < b_r \leq M$ tais que*

$$S_{a_k} = T_{b_k}, \quad 1 \leq k \leq r \quad (3.12)$$

e que r seja o maior possível.

A Definição 10 apresenta um problema que pode ser visualizado da seguinte forma. Suponha que tenhamos duas palavras: o problema consiste em computar a menor quantidade de letras que devem ser removidas das duas palavras para que ambas fiquem iguais.

Por exemplo, suponha que um biotecnólogo precisa comparar duas sequências de DNA. Ele deseja saber a maior sequência de caracteres de DNA que ocorra nas duas sequências. Por exemplo, se ele tem a sequência $A = \text{"TAGC"}$ e $B = \text{"CTCGA"}$. A maior sequência seria **"TG"**.

3.5.1 Solução

Para o subproblema de se computar apenas o tamanho r da maior subsequência comum, uma solução pode ser obtida através de uma função $\text{dp}(i, j)$ que calcula o tamanho da maior subsequência comum considerando apenas o P_i de S e P_j de T . Para isso, calcula-se o tamanho da LCS entre o P_{i-1} de S e o P_j de T , entre o P_i de S e o P_{j-1} de T e para o P_{i-1} de S e o P_{j-1} de T , sendo que neste último caso o tamanho deve ser acrescido em uma unidade se S_i e T_j forem iguais. O maior dentre estes três números será o tamanho da LCS entre o P_i de S e o P_j de T . O caso base ocorre quando algum dos prefixos é vazio: nesse caso o tamanho da LCS entre ele é zero.

Formalmente, seja $\text{dp}(i, j)$ o valor da maior subsequência comum considerando o P_i de S e o P_j de T . Há dois casos base:

1. O índice i é igual a zero. Neste caso a resposta é zero, pois não é possível ter nenhuma subsequência comum se o prefixo de S não existe; e
2. O índice j é igual a zero. Analogamente, nesse caso a resposta também é zero, pois não é possível ter nenhuma subsequência comum se o prefixo de T não existe.

Os casos-base são apresentados na Equação 3.13, onde i ou j são nulos.

Para os demais subproblemas, podemos ter 3 transições possíveis:

1. remover o último caractere de S ; e
2. remover o último caractere de T ; e
3. remover o último caractere de ambos prefixos.

Como a solução do problema deve maximizar a subsequência comum, então a resposta para o subproblema $\text{dp}(i, j)$ será a melhor dentre as três transições, conforme mostra a Equação 3.13.

$$\text{dp}(i, j) = \begin{cases} 0, & \text{se } i = 0 \text{ ou } j = 0 \\ \max(\text{dp}(i-1, j), \text{dp}(i, j-1), \\ \text{dp}(i-1, j-1) + [S_i = T_j]), & \text{se } i > 0 \text{ e } j > 0, \text{ caso contrário.} \end{cases} \quad (3.13)$$

A solução é dada por $\text{dp}(N, M)$. O número de estados é $\Theta(N \times M)$ e cada transição é feita em $\Theta(1)$, de modo que esta solução tem complexidade de tempo e memória $\Theta(N \times M)$.

3.5.2 Implementação

O Código 3.6 apresenta uma possível implementação da solução em C++.

3.6 Problema da soma máxima de sublista contígua

Definição 11 *Seja $\{a_k\}$ uma sequência de N inteiros. O problema da soma máxima de sublista contígua (Maximum Subarray Sum, em inglês) consiste em determinar um par de índices i e j , com $1 \leq i \leq j \leq N$ tais que a soma $\sum_{k=i}^j a_k$ de tal forma que maximize a soma*

$$S = \sum_{k=i}^j a_k \quad (3.14)$$

seja a maior possível.

A Definição 11 apresenta um problema que pode ser visualizado da seguinte forma. Suponha que um investidor tenha os saldos da valorização de suas ações de uma empresa por mês. Ele gostaria de saber qual o intervalo de meses que obteve o maior lucro possível. O problema consiste em informar quais os meses que maximizaram o lucro.

Por exemplo, suponha que ele obteve, a cada mês, os saldos $\{3, -1, 2, -100, 7, -10, 20\}$. Uma solução para o problema seria escolher os três primeiros meses, os quais resultariam

em um lucro de 4. Contudo a melhor solução consiste em escolher os três últimos meses, período no qual ele obteve um lucro de 17.

3.6.1 Solução

Uma solução pode ser obtida por meio de uma função $dp(i)$ que determina a soma máxima no intervalo $[1, i]$. A solução do problema consiste em obter o máximo dentre todas as posições possíveis. Para computar essa função, basta avaliar qual é o valor da soma do intervalo que termina em $i - 1$ ou o valor da sequência em i . O caso base ocorre na primeira posição, na qual o intervalo sempre começa e termina nesse mesmo índice.

Formalmente, seja

$$dp(i) = \max_{1 \leq k \leq i} \sum_{t=k}^i a_t, \quad i = 1, 2, \dots, N.$$

Há um único caso base, que ocorre no primeiro elemento: nesse caso a única opção é escolher o próprio elemento.

O caso base é apresentado na Equação 3.15, quando $i = 1$.

Para os demais subproblemas, é suficiente verificar o maior resultado entre a soma até o índice anterior somado ao elemento atual ou apenas o elemento atual. A transição é dada na Equação 3.15 quando $i \neq 1$.

$$dp(i) = \begin{cases} a_1, & \text{se } i = 1 \\ \max(a_i, dp(i-1) + a_i), & \text{caso contrário.} \end{cases} \quad (3.15)$$

A solução será dada por $\max \{ dp(1), dp(2), \dots, dp(N) \}$. O número de estados é $\Theta(N)$ e cada transição é feita em $\Theta(1)$. Desse modo a complexidade de tempo é $\Theta(N)$. Como é preciso memorizar cada estado, a complexidade de memória também é $\Theta(N)$.

3.6.2 Implementação

O Código 3.7 apresenta uma possível implementação da solução em C++. A versão iterativa é creditada por Kadane [28, pg. 869]. Na solução de Kadane, pela não existência de tabelas, não há um consenso se o algoritmo é resolvido pelo paradigma guloso ou de programação dinâmica.

Código 3.3: Implementação da solução do problema do caixeiro viajante em C++

```
1 const int oo = 1e9;
2 int N, M, S;
3 vector<vector<int>> G;
4 vector<vector<int>> dp;
5
6 // resposta obtida em CaixeiroViajanteRecurso(S, 1 << S)
7 int CaixeiroViajanteRecurso(int idx, int bitmask) {
8
9     if(__builtin_popcount(bitmask) == N) { // caso base: todas as cidades foram visitadas
10         return G[idx][S];
11     }
12
13     auto& ans = dp[idx][bitmask];
14     if(~ans) return ans;
15     ans = oo;
16
17     for(int i = 0; i < N; i++) {
18         if( (bitmask >> i) & 1 ) continue; // ignora se a cidade já foi visitada
19
20         ans = min(ans, CaixeiroViajanteRecurso(i, bitmask | (1 << i)) + G[idx][i]);
21     }
22     return ans;
23 }
24
25 int CaixeiroViajanteIterativo() {
26
27     for(int idx = 0; idx < N; idx++)
28         dp[idx][(1 << N) - 1] = G[idx][S]; // caso base: todas as cidades foram visitadas
29
30     for(int bitmask = (1 << N) - 2; bitmask; bitmask--) {
31         for(int idx = 0; idx < N; idx++) {
32             if((bitmask & idx) == 0) { // é um estado impossível
33                 continue;
34             }
35             for(int i = 0; i < N; i++) {
36                 if( (bitmask >> i) & 1 ) continue; // ignora se a cidade já foi visitada
37
38                 dp[idx][bitmask] = min(ans, dp[i][bitmask | (1 << i)] + G[idx][i]);
39             }
40         }
41     }
42     return dp[S][1 << S];
43 }
```

Fonte: o Autor

Código 3.4: Implementação da solução do problema da maior subsequência crescente em C++

```
1 int N;
2 vector<int> A;
3 vector<int> dp;
4
5 // resposta é o maior valor da função LISRecursiva no intervalo [0, N-1]
6 int LISRecursiva(int idx) {
7
8     auto& ans = dp[idx];
9     if(~ans) return ans;
10    ans = 1;
11
12    for(int i = 0; i < idx; i++) {
13        if(A[i] < A[idx]) {
14            ans = max(ans, LISRecursiva(i) + 1);
15        }
16    }
17    return ans;
18 }
19
20 int LIS() {
21     for(int idx = 0; idx < N; idx++) {
22         dp[idx] = 1; // caso base: a maior sequência terminada em i é no mínimo o próprio elemento em i.
23         for(int i = 0; i < idx; i++) {
24             if(A[i] < A[idx]) {
25                 dp[idx] = max(dp[idx], dp[i] + 1);
26             }
27         }
28     }
29
30     return *max_element(begin(dp), end(dp));
31 }
```

Fonte: o Autor

Código 3.5: Implementação da solução do problema da maior subsequência crescente com busca binária em C++

```
1 int lis(vector<int> const& a) {
2     int n = a.size();
3     const int INF = 1e9;
4     vector<int> d(n+1, INF);
5     d[0] = -INF;
6
7     for (int i = 0; i < n; i++) {
8         int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
9         if (d[l-1] < a[i] && a[i] < d[l])
10             d[l] = a[i];
11     }
12
13     int ans = 0;
14     for (int l = 0; l <= n; l++) {
15         if (d[l] < INF)
16             ans = l;
17     }
18     return ans;
19 }
```

Fonte: CP-Algorithms

Código 3.6: Implementação da solução do problema da maior subsequência comum em C++

```
1 int N, M;
2 string S, T;
3 vector<vector<int>> dp;
4
5 int LCSRecurativa(int idxS, int idxT) {
6
7     if(idxS < 0 || idxT < 0) {
8         return 0;
9     }
10
11     auto& ans = dp[idxS][idxT];
12     if(~ans) return ans;
13
14     int caso1, caso2, caso3;
15
16     // caso em que testa retirar o índice de S
17     caso1 = LCSRecurativa(idxS-1, idxT);
18
19     // caso em que testa retirar o índice de T
20     caso2 = LCSRecurativa(idxS, idxT-1);
21
22     // caso em que testa retirar os dois índices
23     caso3 = LCSRecurativa(idxS-1, idxT-1) + (S[idxS] == T[idxT]);
24
25     return ans = max({caso1, caso2, caso3});
26 }
27
28 int LCS() {
29
30     for(int idxS = 0; idxS < N; idxS++) {
31         for(int idxT = 0; idxT < M; idxT++) {
32             auto& ans = dp[idxS][idxT];
33             int caso1=0, caso2=0, caso3=0;
34
35             // caso em que eu vou pular o idx de S
36             if(idxS - 1 >= 0) {
37                 caso1 = dp[idxS-1][idxT];
38             }
39
40             // caso em que eu vou pular o idx de T
41             if(idxT - 1 >= 0) {
42                 caso2 = dp[idxS][idxT-1];
43             }
44
45             // caso em que eu vou pular os dois
46             // lembrar de checar se os dois são iguais
47             if(idxS - 1 >= 0 && idxT - 1 >= 0) {
48                 caso3 = dp[idxS-1][idxT-1];
49             }
50
51             if(s[idxS] == t[idxT]) { // caso os dois sejam iguais, aumentar resposta do caso 3
52                 caso3++;
53             }
54
55             ans = max({caso1, caso2, caso3});
56         }
57     }
58
59     return dp[N-1][M-1];
60 }
```

Fonte: o Autor

Código 3.7: Implementação da solução da soma máxima de sublista contígua em C++

```
1 int N;
2 vector<int> A;
3 vector<int> dp;
4
5 // a resposta é o máximo da função SomaMaximaSublistaRecurisva no intervalo [0, N-1]
6 int SomaMaximaSublistaRecurisva(int idx) {
7
8     if(idx < 0) return 0;
9
10    auto& ans = dp[idx];
11    if(~ans) return ans;
12
13    return ans = max(SomaMaximaSublistaRecurisva(idx-1) + A[idx], A[idx]);
14 }
15
16 int Kadane() {
17     int atual = 0, melhor = -oo;
18
19     for(int i = 0; i < N; i++) {
20         atual = max(atual+A[i], A[i]);
21         melhor = max(melhor, atual);
22     }
23     return melhor;
24 }
```

Fonte: o Autor

Capítulo 4

DP's Modernas

Neste capítulo serão introduzidas algumas soluções de problemas baseadas em programação dinâmica contendo assuntos mais avançados. Os problemas propostos são temas que aparecem com menos frequência, mas que eventualmente são os problemas que diferenciam os times do topo da tabela em uma competição de programação. O intuito desse capítulo é familiarizar o competidor com essas técnicas, trazendo uma fundamentação para a técnica e também um exemplo de como aplicar a solução.

4.1 SOS DP

Definição 12 *Seja $\mathcal{S}_f : 2^{\mathbb{Z}} \rightarrow \mathbb{Z}$ a função definida por*

$$\mathcal{S}_f(T, M) = \begin{cases} 0, & \text{se } M = \emptyset \\ T(M)f(M) + \sum_{A \subset M} \mathcal{S}_f(T, A), & \text{caso contrário} \end{cases} \quad (4.1)$$

onde $f : 2^{\mathbb{Z}} \rightarrow \mathbb{Z}$ é uma função arbitrária e $T : 2^M \rightarrow [0, 1]$ é um predicado.

O problema da soma sobre subconjuntos (Sum Over Subset – SOS DP, em inglês) consiste em, dado um conjunto finito $M \subset \mathbb{Z}$, computar $\mathcal{S}_f(M, P)$ de forma eficiente.

O problema apresentado na Definição 12 pode ser visualizado da seguinte forma. Suponha que em um jogo, o jogador recebe um conjunto de cartas e a pontuação é determinada pela quantidade de outros jogadores que possuem somente cartas que o jogador também possui. O problema consiste em informar a pontuação de cada jogador.

Por exemplo, suponha que Bob tenha o seguinte conjunto de cartas $B = \{1, 2, 3\}$, Alice tenha $A = \{2, 3, 4\}$ e Caio tenha $C = \{2, 3\}$, logo Bob e Alice terão 1 ponto cada e Caio terá nenhum ponto.

4.1.1 Solução

Para as soluções apresentadas nessa seção, é interessante analisar a relação da representação binária de números inteiros com conjuntos de números inteiros. Por exemplo, o conjunto $\{1, 3, 4\}$ pode ser representado por $2^1 + 2^3 + 2^4 = 26$, logo há uma correspondência entre esse conjunto e o número inteiro 26. As implementações utilizam essa observação para representar um conjunto como um único inteiro. Desta forma, as referências a máscara de *bits* em uma solução corresponde a representação binária de um número que é equivalente a um conjunto.

A primeira solução consiste em verificar todas as máscaras m que são subconjuntos de S . Essa é uma solução de força bruta, pois para cada máscara para a qual será computada a função f , percorre-se por todas as máscaras válidas e verifica-se se é uma sub-máscara. Deste modo, computar f para todos os subconjuntos de tamanho até N , é necessário percorrer todos os 2^N conjuntos possíveis e, para cada um deles, verificar quais dentre eles são subconjuntos, logo a complexidade de tempo é $\Theta(4^N)$. Como é necessário armazenar a resposta para cada conjunto, a complexidade de memória é $\Theta(2^N)$.

Para se otimizar a primeira solução proposta, é possível percorrer apenas os subconjuntos da máscara atual, de tal forma que, para cada máscara sendo testada no momento, serão verificadas apenas suas sub-máscaras. Dessa forma a complexidade de tempo é

$$\Theta\left(\sum_{i=1}^N \binom{N}{i} \times 2^i\right) = \Theta(3^N)$$

A complexidade de memória continua a mesma.

É possível observar que, em ambas soluções, visita-se o mesmo estado múltiplas vezes pois, para duas ou mais máscaras, há uma ou mais sub-máscaras iguais. Assim, pode-se tentar uma solução com programação dinâmica para memorizar os estados. Essa solução propõe uma $dp(i, j)$ que computa a função para a máscara i se é possível alterar o prefixo P_j desta máscara. A transição acontece da seguinte forma: caso o j -ésimo *bit* da máscara esteja desligado, então é somado o valor de P_{j-1} ; caso contrário, o valor é a soma de P_{j-1} mais o valor de P_{j-1} da máscara i com o j -ésimo *bit* desligado. O caso base ocorre quando j é igual a zero: nesse caso, o valor corresponde a $f(i)$. O caso base é apresentado na Equação 4.2 quando $j = 0$.

As outras transições podem ser descritas pela Equação 4.2 quando $j \neq 0$

$$dp(i, j) = \begin{cases} f(i), & \text{se } j = 0 \\ dp(i, j - 1), & \text{se } 2^j \text{ AND } i \\ dp(i, j - 1) + dp(i \text{ XOR } 2^j, j - 1), & \text{caso contrário} \end{cases} \quad (4.2)$$

onde AND é o “e” lógico e XOR é a operação de ou exclusivo, ambos *bit a bit*.

4.1.2 Implementação

O Código 4.1 apresenta uma possível implementação da solução em C++ para as três soluções mostradas.

4.2 Digit DP

Definição 13 *Seja $P : \mathbb{Z} \rightarrow \mathbb{N} \cup \{ 0 \}$ e*

$$f(N) = \sum_{i=1}^N P(i).$$

Considere que, para qualquer inteiro positivo m , com

$$m = d_0 + d_1 \times 10 + d_2 \times 10^2 + \dots + d_k \times 10^k + d_{k+1} \times 10^{k+1},$$

vale que

$$f(m) = g(d_{k+1}, n, P),$$

onde $n = d_0 + d_1 \times 10 + d_2 \times 10^2 + \dots + d_k \times 10^k$. O problema dos dígitos (Digit DP – em inglês) consiste em calcular eficiente a função $f(n)$ para um inteiro positivo n .

O problema apresentado na Definição 13 pode ser visualizado da seguinte forma. Suponha que em um jogo, o ganhador é aquele que escolhe um intervalo de números inteiros de forma que nesse intervalo tenha a maior quantidade de números que satisfaça a propriedade que a soma alternada dos dígitos seja igual a zero.

Por exemplo, o número 1221 satisfaz essa propriedade, pois a soma alternada é $1 - 2 + 2 - 1 = 0$.

4.2.1 Solução

Uma solução pode ser obtida por meio de uma função $\text{dp}(d_{k+1}, n, P)$ que indica quantos números entre $[1, d_{k+1} \times 10^{k+1})$ satisfazem essa propriedade com n sendo o resto dos dígitos a serem computados e P é um predicado que computa quanto esse dígito contribuirá para a resposta. Para computar essa dp é possível testar todos os dígitos x que podem ser usados na i -ésima posição. Dessa forma, para responder quantos números no intervalo $[1, N]$ satisfazem P , é necessário computar $\text{dp}(d_N, n, P)$. Há apenas um caso base, quando $i < 0$: nesse caso, o prefixo é vazio, portanto a resposta é zero. A Equação 4.3 mostra a recorrência da solução.

$$\text{dp}(d_N, n, P) = \begin{cases} 0, & \text{se } d_N < 0 \\ T(P), & \text{se } d_N = 0 \\ \sum_{k=0}^9 \text{dp}(d_{N-1}, n-1, P'), & \text{caso contrário,} \end{cases} \quad (4.3)$$

onde $T(P)$ é uma função que verifica se o número é válido.

A complexidade de tempo é $\Theta(N \times B \times P)$ onde N é a quantidade máxima de dígitos, B é a base numérica (por definição, se utiliza $B = 10$) e P são os estados extras necessários para auxiliar na verificação de $P(i)$. A complexidade de memória é $\Theta(N \times P)$, pois é necessário guardar para cada dígito, quantos números são válidos nesse intervalo com as propriedades de P . Mostrando um caso real, no exemplo dado após a Definição 13, um estado extra que é necessário é a soma alternada dos dígitos e a função P verificaria todos os números que tenham a soma alternada igual a zero.

4.2.2 Implementação

O Código 4.2 apresenta uma possível implementação genérica da solução em C++.

O Código 4.3 apresenta uma possível implementação da solução do exemplo da soma alternada de dígitos em C++.

Código 4.1: Implementações da solução de SOS DP em C++

```

1  int N; // quantidade de elementos dentro de um conjunto
2  vector<int> v; // vector para armazenar os conjuntos
3
4  vector<int> solucao1(vector<int> const& f) { // solução 1: força bruta
5      vector<int> pf(1 << N); // calculando Pf
6      for(int i = 0; i < 1 << N; i++) {
7          for(int j = 0; j < 1 << N; j++) {
8              if(i & j == j) {
9                  pf[i] += f[j];
10             }
11         }
12     }
13     return pf[i];
14 }
15
16 vector<int> solucao2(vector<int> const& f) {
17     vector<int> pf(1 << N);
18     for(int i = 0; i < 1 << N; i++) {
19         pf[i] = f[0];
20         for(int j = i; j; j = (j-1) & i) {
21             pf[i] += f[j];
22         }
23     }
24     return pf;
25 }
26
27 vector<int> solucao3(vector<int> const& f) {
28     vector<vector<int>> pf(1 << N, vector<int>(N));
29     for(int i = 0; i < 1 << N; i++) {
30         pf[i][0] = f[i];
31         for(int j = 0; j < N; j++) {
32             pf[i][j] = pf[i][j-1];
33             if(i & (1 << j))
34                 pf[i][j] += pf[i ^ (1 << j)][j-1];
35         }
36     }
37     return pf;
38 }
39
40 vector<int> solucao3memoriaOtimizada(vector<int> const& f) {
41     vector<int> pf(1 << N);
42
43     for(int i = 0; i < 1 << N; i++)
44         pf[i] += f[i];
45
46     for(int i = 0; i < 1 << N; i++)
47         for(int j = 0; j < N; j++)
48             if(i & (1 << j))
49                 pf[i] += pf[i ^ (1 << j)];
50     return pf;
51 }

```

Fonte: o Autor

Código 4.2: Implementação da solução de DP de dígito em C++

```
1 string T; // calcular todos inteiros até T que estejam no conjunto A
2 int dp[pos][limite][...args]; // tabela para armazenar a resposta
3
4 int f(int i, bool limite, ...args) { // deve ser inicializado a tabela dp com -1
5     if (i == T.size()) return ...args; // aqui se utiliza os estados de ...args para verificar se o número está em A
6
7     auto& ans = dp[i][limite][...args];
8     if(ans != -1) return ans;
9     ans = 0;
10    int ultimo_digito = (limite ? T[i] - '0' : 9); // verifica até que dígito precisa-se testar
11    for (int k = 0; k <= ultimo_digito; k++) {
12        bool novo_limite = limite && (k == ultimo_digito); // verifica se deve ser passado o limite para a próxima chamada
13        ans += f(i + 1, novo_limite, ...novo_args); // precisa atualizar os args baseado em k
14    }
15
16    return ans;
17 }
```

8

Fonte: o Autor

Código 4.3: Implementação da solução de exemplo de DP de dígito em C++

```
1 // código para resolver o problema de quantos números tem a soma alternada igual a zero
2 // o conjunto A pertence todos os números cuja soma alternada de dígitos é zero
3 string T; // calcular todos inteiros até T que estejam no conjunto A
4 map<array<int, 3>, int> dp; // utilizando map para não precisar lidar com a soma negativa
5
6 int f(int i, bool limite, int soma) { // a resposta é computada em f(0, 1, 0)
7     if (i == T.size()) return soma == 0; // verificar se a soma é zero
8
9     if(dp.find({i, limite, soma}) != dp.end()) return dp[{i, limite, soma}];
10    int ans = 0;
11    int ultimo_digito = (limite ? T[i] - '0' : 9); // verifica até que dígito precisa-se testar
12    for (int k = 0; k <= ultimo_digito; k++) {
13        bool novo_limite = limite && (k == ultimo_digito); // verifica se deve ser passado o limite para a próxima chamada
14        ans += f(i + 1, novo_limite, soma + (i%2 ? k : -k)); // verificar a paridade do i-ésimo dígito e somar ou subtrair com k
15    }
16
17    return dp[{i, limite, soma}] = ans;
18 }
```

Fonte: o Autor

Capítulo 5

Aplicações de programação dinâmica na solução de problemas

Este capítulo apresenta problemas de juízes online, principalmente do AtCoder¹, CSES² e Codeforces³, cujas soluções requerem o uso de técnicas de programação dinâmica apresentadas nesse trabalho. Após a contextualização e ilustração de alguns exemplos, será descrita a solução do problema. Esta abordagem tem como objetivo mostrar a importância da técnica de programação dinâmica utilizada na solução, bem como ajudar o leitor a se familiarizar com a referida técnica. É fortemente recomendado que o leitor tente resolver as questões antes de ler a seção de solução.

5.1 AtCoder Educational DP Contest – Knapsack 2

Resumo do problema⁴: existem N ($1 \leq N \leq 100$) itens, cada item i tem uma massa w_i ($1 \leq w_i \leq W$) e um valor v_i ($1 \leq v_i \leq 10^3$). Taro tem uma mochila que comporta até W ($1 \leq W \leq 10^9$) de massa e o problema consiste em escolher alguns itens que caibam na mochila e maximizem o valor total.

Exemplo: $N = 3$, $W = 8$. As massas dos itens são $\{3, 4, 5\}$ e o valor dos itens são $\{30, 50, 60\}$.

1. Ao escolher os dois primeiros itens, a massa total dos itens é 7, que é inferior a capacidade máxima W , e o valor total dos itens é 80.
2. Porém, a melhor resposta é escolher o primeiro e o último itens. A massa total é 8 e o valor total dos itens é 90, que é o máximo possível.

¹<https://atcoder.jp>

²<https://cses.fi/problemset/list/>

³<https://codeforces.com>

⁴https://atcoder.jp/contests/dp/tasks/dp_e

5.1.1 Solução

A maior observação é que esse é o mesmo problema da mochila tradicional (Seção 3.1), porém os limites de peso e valor estão trocados⁵, então é necessário modelar a função de uma forma diferente. Uma solução pode ser obtida através de uma função $dp(i, j)$ que computa a menor massa utilizada para conseguir j de valor usando um subconjunto dos i primeiros itens. O caso base ocorre quando o subconjunto é vazio: nesse caso a resposta é 0, pois não podemos pegar nenhum item. Para as transições, é necessário verificar a menor massa que é necessária para chegar no valor j entre tentar incluir ou não o i -ésimo elemento.

Para encontrar a resposta, é necessário verificar para todos os valores se o menor peso para obter esse valor é menor que a capacidade da mochila e escolher o maior valor entre eles. É necessário um estado de tamanho N para cada prefixo e um estado de tamanho $N \times |V|$ para computar o valor máximo de cada prefixo, uma transição constante de forma que a complexidade de tempo e memória é $\Theta(N^2 \times |V|)$, em que $|V|$ é o valor máximo de um item.

5.1.2 Implementações

O Código 5.1 apresenta uma possível implementação recursiva da solução em C++⁶.

O Código 5.2 apresenta uma possível implementação iterativa da solução em C++⁷.

5.2 AtCoder Educational DP Contest – Frog 2

Resumo do problema⁸: existem N ($1 \leq N \leq 10^5$) pedras, cada pedra i ($1 \leq i \leq N$) tem uma altura h_i ($1 \leq h_i \leq 10^4$). Um sapo está parado na pedra 1. Ele pode pular até K ($1 \leq K \leq 100$) pedras para a frente, com um custo de $|h_i - h_{i+k}|$, onde i é a pedra em que ele está. Encontre o menor custo possível para o sapo chegar na pedra N .

Exemplo: $N = 5$, $K = 3$. As alturas das pedras são $\{10, 30, 40, 50, 20\}$.

1. O sapo pode pular para as seguintes pedras $[1, 4, 5]$, com custo $|h_1 - h_4| + |h_4 - h_5| = 70$.
2. Porém, a melhor resposta é $[1, 2, 5]$. O custo é $|h_1 - h_2| + |h_2 - h_5| = 30$.

⁵Vale notar que ao tentar utilizar a solução clássica da mochila, não é possível resolver o problema por questões de tempo e memória, dado que $N \times W$ é um valor muito grande para ser computado dentro dos limites de memória e tempo de competições tradicionais.

⁶<https://atcoder.jp/contests/dp/submissions/60924714>

⁷<https://atcoder.jp/contests/dp/submissions/60924818>

⁸https://atcoder.jp/contests/dp/tasks/dp_b

5.2.1 Solução

A observação para esse problema é que ele pode ser visualizado como uma variação do problema da moeda (Seção 3.2). Cada pedra é o valor do troco e os K pulos são as moedas. Dessa forma verificamos o menor custo para atingir a N -ésima pedra se pudermos usar qualquer um dos K pulos.

Uma solução pode ser obtida por meio de uma função $dp(i)$ que computa o menor custo para chegar na i -ésima pedra. O caso base ocorre quando o sapo está na primeira pedra: neste caso, a resposta é zero. Existem K transições: uma para cada possível destino j , computa-se $dp(i - j) + |h_i - h_j|$ e opta-se pelo menor dentre todos os valores obtidos.

A resposta corresponde a $dp(N)$: o menor custo para chegar na N -ésima pedra. Como existem N estados e K transições, a complexidade de tempo é $\Theta(NK)$ e a complexidade de espaço é $\Theta(N)$.

5.2.2 Implementações

O Código 5.3 apresenta uma possível implementação recursiva da solução em C++⁹.

O Código 5.4 apresenta uma possível implementação iterativa da solução em C++¹⁰.

5.3 CSES DP – Towers

Resumo do problema¹¹: existem N ($1 \leq N \leq 2 \times 10^5$) cubos em sequência, cada cubo i ($1 \leq i \leq N$) tem um lado h_i ($1 \leq h_i \leq 10^9$). Deve-se empilhar os cubos para formar uma torre, porém o cubo de cima deve ser menor que o cubo de baixo. Ao processar os cubos em ordem, é possível empilhar o cubo atual em cima de outro ou criar uma nova torre. O problema consiste em minimizar a quantidade de torres.

Exemplo: $N = 5$. Os lados do cubo são $\{3, 8, 2, 1, 5\}$.

1. Pode-se escolher 3 torres de tal forma que os índices $\{1, 2, 3\}$ sejam a base das torres.
2. Porém, a melhor resposta é $\{1, 2\}$. A primeira torre será composta pelos elementos de índice $\{1, 3, 4\}$ e a segunda torre será composta pelos elementos de índice $\{2, 5\}$.

⁹<https://atcoder.jp/contests/dp/submissions/61467955>

¹⁰<https://atcoder.jp/contests/dp/submissions/61467931>

¹¹<https://cses.fi/problemset/task/1073>

5.3.1 Solução

A observação para esse problema é que ele pode ser visualizado como uma variação da LIS (Seção 3.4). Nesse problema, deve-se minimizar a quantidade de subsequências decrescentes que cobrem a lista dada e é possível provar que a solução deste problema é igual a maior subsequência não-decrescente. Primeiramente, pode-se afirmar que a resposta é no mínimo a maior subsequência não-decrescente. De fato, se a resposta for menor, então dois elementos da maior subsequência não-decrescente devem estar na mesma subsequência, o que é impossível, pois esses dois elementos não formariam uma subsequência decrescente. Agora que há um limite para a resposta que é, pelo menos, o tamanho da maior subsequência não-decrescente, deve-se provar que a resposta corresponde exatamente a esse limite.

Essa prova é uma tradução livre do USACO Guide [29]. A motivação para a prova se baseia na observação de um algoritmo guloso para esse problema. Esse algoritmo guloso deve processar todo elemento x , da esquerda para a direita, e adicioná-lo à torre cujo elemento no topo seja o menor possível dentre aqueles que são maiores que x . Caso não exista tal elemento, deve ser criada uma nova torre com base igual a x . Este algoritmo minimiza a quantidade de subsequências decrescentes.

Suponha que f_i seja a maior subsequência não decrescente que termina em h_i . Seja L a lista formada por todos os elementos h_i tal que $f_i = t$ para um valor fixo t . Logo L deve ser uma subsequência decrescente, caso contrário, o valor t seria diferente para dois elementos distintos. Logo, se A é o tamanho da maior subsequência não decrescente, cada elemento em A faz parte de uma subsequência decrescente distinta, assim h é coberto com $|A|$ subsequências decrescentes, onde $|A|$ é o tamanho de A . Então basta computar a maior subsequência não decrescente para obter a resposta. Esse é um caso especial do Teorema de Dilworth [30].

Com esta prova, a solução de LIS é utilizada para resolver este problema, de forma que a complexidade de tempo é $\Theta(N \times \log N)$ e a complexidade de memória $\Theta(N)$.

5.3.2 Implementação

O Código 5.5 apresenta uma possível implementação da solução em C++¹².

¹²<https://cses.fi/problemset/result/12056807/>

5.4 Educational Codeforces Round 123 – Increasing Subarray Sums

Resumo do problema¹³: dada uma sequência a_k de inteiros de tamanho N ($1 \leq N \leq 5000$), um inteiro x ($0 \leq x \leq 10^5$) e uma função $f(k)$ que é a soma máxima entre todas as sublistas $b_{ij} = \{ a_i, a_{i+1}, \dots, a_j \}$ de a após aplicar a seguinte operação: adicionar x em k elementos distintos. Em termos mais precisos

$$f(k) = \max_{\substack{s_k \subset [1, N] \\ 1 \leq i \leq j \leq N}} \sum_{t=i}^j a_t \times \delta_{s_k}(i)$$

onde $\delta_{s_k}(i) = 1$, se $i \in s_k$, e $\delta_{s_k}(i) = 0$, caso contrário, e s_k é um subconjunto de inteiros do intervalo $[1, N]$ com $|s_k| = k$. O problema consiste em computar $f(k)$ para todo k entre 0 e N .

Exemplo: $a = \{-6, -1, -2, 4, -6, -1, -4, 4, -5, -4\}$ e $x = 2$.

1. Quando $k = 1$, pode-se aplicar a operação no primeiro elemento e aplicar o algoritmo para encontrar a maior soma de sublista contínua, cujo resultado é 4.
2. Porém o melhor caso para $k = 1$ é aplicar a operação no quarto elemento. Dessa forma, após aplicar o algoritmo da maior soma de sublista contínua, temos resultado igual a 6, que é o maior possível dentre todas as possibilidades.
3. Quando $k = 3$, o melhor é aplicar a operação nos elementos de índices 2, 3, 4: neste caso, a resposta é 7.

5.4.1 Solução

Essa é uma variação do problema de soma máxima de sublista contínua (Seção 3.6). Pelos limites do problema, pode-se utilizar uma solução de programação dinâmica para cada k . Para cada k , pode-se modelar a função da seguinte forma: $dp(i, k)$ é a maior soma terminando em i ao aplicar a operação k vezes. A resposta para o problema será o maior valor dentre todos os prefixos.

Existem dois casos bases. O primeiro ocorre no prefixo de tamanho 1: neste caso, a resposta é o valor do primeiro elemento e, caso ainda seja possível aplicar a operação, o valor x deve ser adicionado à resposta. O segundo caso base ocorre quando não se pode aplicar a operação: nesse caso, basta utilizar a soma de sublista contínua apresentada na Seção 3.15.

¹³<https://codeforces.com/problemset/problem/1644/C>

Há três transições possíveis: a maior soma terminar no mesmo índice, estender a soma do índice anterior sem aplicar a operação e estender a soma do índice anterior e aplicar a operação anterior. Como o número de índices distintos e de operações possíveis são proporcionais a N , e como há apenas três transições, então a complexidade de tempo é $\Theta(N^2)$. Como é necessário guardar o valor para cada estado, a complexidade de memória também é $\Theta(N^2)$.

5.4.2 Implementação

O Código 5.6 apresenta uma possível implementação recursiva da solução em C++¹⁴.

5.5 SPOJ – MC

Resumo do problema¹⁵: existem duas *strings* S , T . O custo para deletar um elemento de S é 15 e o custo para deletar um elemento de T é 30. O problema consiste em minimizar o custo para que S seja igual a T . Deve-se responder várias questões, o fim da entrada ocorre quando S é igual a '#'.

Exemplo: $S = \text{"abcd"}$ e $T = \text{"acdb"}$.

1. Pode-se deletar os três últimos caracteres de cada, tendo um custo de $3 \times 15 + 3 \times 30 = 135$.
2. Porém, a melhor resposta é deletar o segundo elemento de S e o último elemento de T , com um custo mínimo de 45.

5.5.1 Solução

Esse problema pode ser visualizado como uma variação da LCS (Seção 3.5). Devemos modificar a função $\text{dp}(i, j)$ para minimizar o custo descrito pelo problema ao invés de computar o tamanho da maior subsequência comum.

Uma solução pode ser descrita por meio de uma função $\text{dp}(i, j)$ que computa o custo mínimo para que o prefixo que termina em S_i e o prefixo que termina em T_j sejam iguais. O caso base ocorre quando um prefixo for vazio. Existem três casos: o primeiro caso ocorre quando ambos prefixos são vazios, de modo que o custo é zero; o segundo caso ocorre quando apenas o prefixo de S é vazio, nesse caso o custo é a quantidade de caracteres em T vezes 30; o terceiro caso é análogo ao segundo, porém agora o custo é a quantidade de caracteres em S vezes 15. Há também três transições: a primeira consiste em deletar o

¹⁴<https://codeforces.com/contest/1644/submission/301307965>

¹⁵<https://www.spoj.com/problems/MC/>

último caractere de S ; a segunda é deletar o último caractere de T ; a última é processar os últimos caracteres das duas *strings*: se ambos forem iguais, o custo será zero, caso contrário ambos devem ser removidos, com custo $30 + 15 = 45$.

Como o problema é apenas uma variação da LCS, as complexidades de tempo e memória são as mesmas. Considerando que existam C casos de teste, a complexidade da solução é de $\Theta(C \times N \times M)$, onde N é o tamanho de S e M é o tamanho de T .

5.5.2 Implementação

O Código 5.7 apresenta uma possível implementação recursiva da solução em C++¹⁶.

5.6 AtCoder Begginer Contest 180 – Traveling Salesman among Aerial Cities

Resumo do problema¹⁷: existem N ($1 \leq N \leq 17$) cidades em um plano tridimensional. O custo para ir de uma cidade nas coordenadas (a, b, c) até a cidade na coordenada (x, y, z) é $|a - x| + |b - y| + \max(0, z - c)$. O problema consiste em começar pela cidade 1, visitar todas as cidades pelo menos uma vez e terminar na cidade 1.

Exemplo: existem $N = 2$ cidades. A primeira cidade está na coordenada $(0, 0, 0)$ e a segunda está na coordenada $(1, 2, 3)$.

1. O único caminho possível é começar em 1, ir para 2 e voltar para 1. Dessa forma, o custo é $|0 - 1| + |0 - 2| + \max(0, 3 - 0) + |1 - 0| + |2 - 0| + \max(0, 0 - 3) = 9$.

5.6.1 Solução

Para este problema, deve-se observar que ele é bastante similar ao clássico problema do caixeiro viajante (Seção 3.3). A principal diferença é que o custo das arestas é dado pela função apresentada no enunciado.

Uma solução pode ser obtida por meio de uma função $\text{dp}(i, \text{bitmask})$ que computa o menor caminho para chegar em i caso as cidades em bitmask tenham sido visitadas. A modelagem é semelhante ao problema original, mudando apenas a função do custo das arestas. De fato, uma solução possível é modelar esse problema como um grafo completo, criando arestas entre todos nós com o custo da função descrita pelo problema e resolver o problema do caixeiro viajante original. As complexidades de tempo e memória permanecem as mesmas do problema original.

¹⁶<https://www.spoj.com/status/MC,eyz/>

¹⁷https://atcoder.jp/contests/abc180/tasks/abc180_e

5.6.2 Implementação

O Código 5.8 apresenta uma possível implementação recursiva da solução em C++¹⁸.

5.7 AtCoder Educational DP Contest – Digit Sum

Resumo do problema¹⁹: contar quantos números no intervalo $[1, K]$ ($1 \leq K \leq 10^{10000}$) satisfazem a propriedade que a soma dos dígitos é divisível por D ($1 \leq D \leq 100$). Como essa quantidade é grande, imprima a resposta módulo $10^9 + 7$.

Exemplo: $K = 30$ e $D = 4$. Nesse exemplo, há 6 números que satisfazem a propriedade, sendo eles: 4, 8, 13, 17, 22, 26.

1. O número 22 atende a propriedade, pois $2 + 2 = 4$.
2. O número 17 também atende a propriedade, pois $1 + 7 = 8$.
3. O número 11, por outro lado, não atende a propriedade, pois $1 + 1 = 2$, e 2 não é divisível por 4.

5.7.1 Solução

Pela contextualização do problema, pode-se resolver com uma abordagem de DP de dígitos (Seção 4.2). O problema pode ser resolvido ao modelar uma função $f(idx, sum, lead)$, que computa quantos números até $x \times 10^{idx}$ satisfazem a propriedade de que a soma dos dígitos deixa resto sum módulo D , onde x é 9 caso $lead$ seja falso, caso contrário x é o idx -ésimo dígito de K . Existem 10 transições, uma para cada dígito possível, onde se computa quantos números até aquele dígito resultam em um resto igual a $sum - curD$ módulo D , onde $curD$ é o dígito da transição. O caso base ocorre quando se está no primeiro dígito, neste caso, verifica-se quais dos dígitos tem o mesmo resto de sum módulo D . A função f computa a propriedade no intervalo $[0, K]$, então para obter a solução para o problema é necessário diminuir 1 de $f(0, 0, 1)$.

Existem três estados, um estado para a quantidade de dígitos K , outro estado para o resto da soma módulo D e um estado para saber se está sendo processado o último número; e 10 transições. Dessa forma a complexidade de tempo e espaço é $\Theta(|K| \times D)$, onde $|K|$ é a quantidade de dígitos de K .

¹⁸<https://atcoder.jp/contests/abc180/submissions/40524269>

¹⁹https://atcoder.jp/contests/dp/tasks/dp_s

5.7.2 Implementação

O Código 5.9 apresenta uma possível implementação recursiva da solução em C++²⁰.

5.8 CSES Additional Problems – Bit Problem

Resumo do problema²¹: dada uma lista a de inteiros de tamanho N ($1 \leq N \leq 2 \times 10^5$), o problema consiste em responder esses três subproblemas para cada número x na lista:

1. Quantos números y existem tal que $x \vee y = x$?
2. Quantos números y existem tal que $x \wedge y = x$?
3. Quantos números y existem tal que $x \wedge y \neq 0$?

onde $\vee$ é o 'ou lógico' binário e $\wedge$ é o 'e lógico' binário.

Exemplo: $a = \{3, 7, 2, 9, 2\}$.

1. Para o primeiro elemento da lista, existem três elementos que satisfazem a primeira propriedade: os índices 1, 3, 5. Pois $3 \vee 3 = 3$ e $3 \vee 2 = 3$.
2. Para o primeiro elemento da lista, existem dois elementos que satisfazem a segunda propriedade: os índices 1, 2. Pois $3 \wedge 3 = 3$ e $3 \wedge 7 = 3$.
3. Para o primeiro elemento da lista, todos os elementos na lista satisfazem a terceira propriedade. Pois $3 \wedge 3 \neq 0$, $3 \wedge 2 \neq 0$, $3 \wedge 7 \neq 0$ e $3 \wedge 9 \neq 0$.

5.8.1 Solução

Esse problema pode ser dividido em três tarefas. Cada tarefa pode ser visualizada como uma variação do problema da soma de subconjuntos (Seção 4.1). Para resolver esse problema, deve-se utilizar a técnica de visualizar o número como um conjunto na base 2, onde o 1 representa que o elemento pertence ao conjunto e o 0 que não pertence.

A primeira tarefa é contar quantos subconjuntos de x estão presentes na lista. Para isso, pode-se utilizar a abordagem clássica desse problema para resolver essa tarefa. Cria-se um histograma dos elementos e é construída a solução baseado nesse histograma.

A segunda tarefa pode ser visualizado como contar quantos super conjuntos²² de x tem na lista. Dessa forma, a abordagem é semelhante à tarefa anterior: a única mudança

²⁰<https://atcoder.jp/contests/dp/submissions/61881833>

²¹<https://cses.fi/problemset/task/1654>

²²Super conjunto são todos os conjuntos que contém x .

é que a transição deve ser feita quando o elemento não pertence ao conjunto, de tal forma que deve-se considerar incluí-lo ou não na resposta.

Para a terceira tarefa, ela pode ser resolvida a partir do complemento do problema, ou seja, determinar quantos elementos y existem tal que $x \wedge y = 0$. Ao observar o problema dessa forma, se torna bem mais simples a resolução, pois pode-se contar quantos elementos satisfazem esse complemento e a resposta consistirá em todos os demais elementos. Para resolver, basta contar quantos subconjuntos existem no complemento do conjunto associado a x .

5.8.2 Implementação

O Código 5.10 apresenta uma possível implementação da solução em C++²³.

5.9 Lista de problemas

Segue uma lista de problemas para o leitor se familiarizar com as técnicas descritas neste trabalho e outros problemas envolvendo programação dinâmica.

1. CSES — Book Shop²⁴
2. CSES — Book Shop II²⁵
3. Codeforces — LCS on Permutations²⁶
4. CSES — Increasing Subsequence²⁷
5. AtCoder — LIS on Tree²⁸
6. CSES — Towers²⁹
7. SPOJ — HowManyLis³⁰
8. SPOJ — One X LIS³¹
9. SPOJ — Supernumbers in a permutation³²

²³<https://cses.fi/problemset/result/11798828/>

²⁴<https://cses.fi/problemset/task/1158/>

²⁵<https://cses.fi/problemset/task/1159>

²⁶<https://codeforces.com/gym/102951/problem/C>

²⁷<https://cses.fi/problemset/task/1145>

²⁸https://atcoder.jp/contests/abc165/tasks/abc165_f

²⁹<https://cses.fi/problemset/task/1073>

³⁰<https://www.spoj.com/problems/HMLIS/>

³¹<https://www.spoj.com/problems/ONEXLIS/>

³²<https://www.spoj.com/problems/SUPPER/>

10. Codeforces — Big Maximum Sum³³
11. Codeforces — Maximum Subarray³⁴
12. AtCoder — LCS³⁵
13. Codeforces — Lucky Common Subsequence³⁶
14. Codeforces — Consecutive Subsequence³⁷
15. AtCoder — Negative Traveling Salesman³⁸
16. CSES — Counting Numbers³⁹
17. AtCoder — Digit Products⁴⁰

³³<https://codeforces.com/contest/75/problem/D>

³⁴<https://codeforces.com/problemset/problem/1796/D>

³⁵https://atcoder.jp/contests/dp/tasks/dp_f

³⁶<https://codeforces.com/problemset/problem/346/B>

³⁷<https://codeforces.com/problemset/problem/977/F>

³⁸https://atcoder.jp/contests/abc338/tasks/abc338_f

³⁹<https://cses.fi/problemset/task/2220>

⁴⁰https://atcoder.jp/contests/abc208/tasks/abc208_e?lang=en

Código 5.1: Implementação recursiva da solução do problema Knapsack 2 em C++

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const int oo = 2e9;
6 const int MAXV = 1e5+10;
7 int N, W;
8 vector<int> peso, valor;
9 vector<vector<long long>> tab;
10
11 long long dp(int i, int j) {
12
13     if(i < 0) {
14         if(j == 0)
15             return 0;
16         return oo;
17     }
18
19     auto& ans = tab[i][j];
20     if(ans != -1) return ans;
21     ans = oo;
22
23     // nao incluir o elemento atual
24     ans = dp(i-1, j);
25
26     if(j - valor[i] >= 0)
27         ans = min(ans, dp(i-1, j - valor[i]) + peso[i]);
28     return ans;
29 }
30
31 int main() {
32     cin.tie(0)->sync_with_stdio(false);
33
34     cin >> N >> W;
35     peso.resize(N);
36     valor.resize(N);
37     tab.resize(N, vector<long long>(MAXV, -1));
38     for(int i = 0; i < N; i++)
39         cin >> peso[i] >> valor[i];
40
41     int ans = 0;
42     for(int i = MAXV-1; i; i--) {
43         if(dp(N-1, i) <= W) {
44             ans = i;
45             break;
46         }
47     }
48
49     cout << ans << endl;
50
51 }
```

Fonte: o Autor

Código 5.2: Implementação iterativa da solução do problema Knapsack 2 em C++

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const int MAXV = 1e5+8;
6 const int oo = 2e9;
7
8 int main() {
9     cin.tie(0)->sync_with_stdio(false);
10
11     int N, W;
12     cin >> N >> W;
13     vector<int> peso(N), valor(N);
14     vector<vector<long long>> tab(N+1, vector<long long>(MAXV, oo));
15     for(int i = 0; i < N; i++)
16         cin >> peso[i] >> valor[i];
17
18     tab[0][0] = 0;
19
20     for(int i = 1; i <= N; i++) {
21         for(int j = 0; j < MAXV; j++) {
22             tab[i][j] = min(tab[i][j], tab[i-1][j]);
23
24             if(j - valor[i-1] >= 0)
25                 tab[i][j] = min(tab[i][j], tab[i-1][j - valor[i-1]] + peso[i-1]);
26         }
27     }
28
29     int ans = 0;
30     for(int i = MAXV-1; i; i--) {
31         if(tab[N][i] <= W) {
32             ans = i;
33             break;
34         }
35     }
36     cout << ans << endl;
37 }
```

Fonte: o Autor

Código 5.3: Implementação recursiva da solução do problema Frog 2 em C++

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  const int oo = 2e9;
6
7  int N, K;
8  vector<int> h, dp;
9
10 int f(int i) {
11     if(i == 1) {
12         return 0;
13     }
14
15     auto& ans = dp[i];
16     if(ans != -1) return ans;
17     ans = oo;
18
19     for(int j = 1; j <= K; j++) {
20         if(i - j > 0) {
21             ans = min(ans, f(i-j) + abs(h[i-1] - h[i-j-1]));
22         }
23     }
24     return ans;
25 }
26
27 int main() {
28     cin.tie(0)->sync_with_stdio(false);
29
30     cin >> N >> K;
31     h.resize(N);
32     dp.resize(N+1, -1);
33     for(auto& it: h) cin >> it;
34     cout << f(N) << endl;
35 }
```

Fonte: o Autor

Código 5.4: Implementação iterativa da solução do problema Frog 2 em C++

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const int oo = 2e9;
6
7 int main() {
8     cin.tie(0)->sync_with_stdio(false);
9
10    int N, K;
11    cin >> N >> K;
12    vector<int> h(N);
13    for(auto& it: h) cin >> it;
14
15    vector<int> dp(N+1, oo);
16    dp[1] = 0; // caso base: primeira pedra
17    for(int i = 2; i <= N; i++) {
18        for(int j = 1; j <= K; j++) {
19            if(i-j <= 0) break; // ele vai pular de uma pedra que nao existe
20            dp[i] = min(dp[i], dp[i-j] + abs(h[i-1] - h[i-j-1]));
21        }
22    }
23
24    cout << dp[N] << endl;
25 }
```

Fonte: o Autor

Código 5.5: Implementação da solução do problema *Towers* em C++

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     cin.tie(0)->sync_with_stdio(false);
7
8     const int oo = 1e9 + 10;
9
10    int n;
11    cin >> n;
12    vector<int> d(n+1, oo);
13    d[0] = -oo;
14
15    for(int i = 0; i < n; i++) {
16        int a; cin >> a;
17
18        auto l = upper_bound(begin(d), end(d), a) - begin(d);
19
20        if(d[l-1] <= a && a < d[l])
21            d[l] = a;
22    }
23
24    int ans = 0;
25    for(int i = 0; i <= n; i++)
26        if(d[i] < oo)
27            ans = i;
28
29    cout << ans << endl;
30 }
```

Fonte: o Autor

Código 5.6: Implementação recursiva da solução do problema *Increasing Subarray Sums* em C++

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  const int MAXN = 5005;
6  int n, x;
7  vector<int> v(MAXN);
8  int tb[MAXN][MAXN];
9  bool vis[MAXN][MAXN];
10
11 int dp(int i, int j) {
12
13     if(i == 0) { // caso base para o primeiro índice
14         return v[i] + (j ? x : 0);
15     }
16
17     if(j == 0) // se j for 0, então é um kadane
18         return max(dp(i-1, j) + v[i], v[i]);
19
20     if(vis[i][j]) return tb[i][j];
21     vis[i][j] = 1;
22
23     return tb[i][j] = max({
24         dp(i-1, j) + v[i],
25         dp(i-1, j-1) + v[i]+x,
26         v[i]+x
27     });
28 }
29
30 int main() {
31     cin.tie(0)->sync_with_stdio(false);
32
33     int t; cin >> t; while(t--) {
34         cin >> n >> x;
35         for(int i = 0; i < n; i++) cin >> v[i];
36
37         for(int i = 0; i < n; i++)
38             for(int j = 0; j <= n; j++)
39                 vis[i][j] = 0;
40
41         for(int j = 0; j <= n; j++) {
42             int ans = 0;
43             for(int i = 0; i < n; i++) {
44                 ans = max(ans, dp(i, j)); // testando o máximo que termina em cada índice i aplicando j operações
45             }
46             cout << ans << ' ';
47         }
48         cout << '\n';
49     }
50 }
```

Fonte: o Autor

Código 5.7: Implementação recursiva da solução do problema MC em C++

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  const int oo = 1e5;
6  const int MAXN = 1010;
7  string s, t;
8  vector<vector<int>> dp(MAXN, vector<int>(MAXN));
9
10 int f(int i, int j) {
11     if(i < 0 && j < 0) { // caso base: ambos tamanhos são vazios
12         return 0;
13     }
14
15     if(i < 0) { // caso base: tamanho de S é vazio, precisa apagar todos os caracteres de T
16         return (j+1) * 30;
17     }
18
19     if(j < 0) { // caso base: tamanho de T é vazio, precisa apagar todos os caracteres de S
20         return (i+1) * 15;
21     }
22
23     auto& ans = dp[i][j];
24     if(ans != -1) return ans;
25     ans = oo;
26
27     return ans = min({f(i-1, j) + 15, f(i, j-1) + 30, f(i-1, j-1) + (s[i] != t[j] ? 45 : 0)});
28 }
29
30
31 int main() {
32     cin.tie(0)->sync_with_stdio(false);
33
34     while(cin >> s, s != "#") {
35         cin >> t;
36         for(int i = 0; i < s.size(); i++)
37             for(int j = 0; j < t.size(); j++)
38                 dp[i][j] = -1;
39         cout << f(s.size() - 1, t.size() - 1) << '\n';
40     }
41 }
```

Fonte: o Autor

Código 5.8: Implementação recursiva da solução do problema Traveling Salesman among Aerial Cities em C++

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 using iii = tuple<int,int,int>;
6 const int oo = 1e9;
7 const int MAXN = 18;
8 int n;
9 vector<iii> v;
10 int tb[MAXN][(1 << MAXN)];
11
12 int dp(int idx, int bit) {
13
14     auto [a1, b1, c1] = v[idx];
15     if(__builtin_popcount(bit) == n) { // caso base: visitou todas as cidades
16         auto [a2, b2, c2] = v[0];
17         return abs(a1-a2) + abs(b1-b2) + max(0, c2-c1); // custo de ir da última cidade para a primeira.
18     }
19
20     auto& res = tb[idx][bit];
21     if(!res) return res;
22     res = oo;
23
24     for(int i = 0; i < n; i++) {
25         auto [a2, b2, c2] = v[i];
26         if(i == idx) continue; // ignora caso seja a cidade sendo processada atualmente
27         if((bit & (1 << i)) != 0) continue; // ignora caso a cidade já foi processada
28
29         bit |= (1 << i); // inserir a cidade i na bitmask
30         res = min(res, abs(a1-a2) + abs(b1-b2) + max(0, c2-c1) + dp(i, bit)); // verificando o menor custo
31         bit ^= (1 << i); // remover a cidade i da bitmask
32     }
33
34     return res;
35 }
36
37 int main() {
38     cin.tie(0)->sync_with_stdio(false);
39
40     memset(tb, -1, sizeof tb);
41     cin >> n;
42     v.resize(n);
43     for(auto& [a, b, c]: v)
44         cin >> a >> b >> c;
45
46     cout << dp(0, 1) << endl;
47 }
```

Fonte: o Autor

Código 5.9: Implementação recursiva da solução do problema Digit Sum em C++

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const int MAXK = 10010;
6 const int MAXD = 102;
7 const int MOD = 1e9 + 7;
8 string k;
9 int d;
10 vector<vector<vector<int>>> dp;
11
12 // conta quantos números até 10^idx existem tal que a soma dos dígitos módulo D é sum
13 int f(int idx, int sum, bool lead) {
14
15     if(idx == size(k) - 1) { // caso base: casa das unidades
16         int ans = 0;
17         for(int curDigit = 0; curDigit <= (lead ? k[idx] - '0' : 9); curDigit++) {
18             if(curDigit % d == sum) ans++;
19         }
20         return ans;
21     }
22
23     auto& ans = dp[idx][sum][lead];
24     if(ans != -1)
25         return ans;
26
27     ans = 0;
28
29     for(int curDigit = 0; curDigit <= (lead ? k[idx] - '0' : 9); curDigit++) {
30         ans += f(idx+1, ((sum - curDigit) % d + d) % d, (lead && curDigit == k[idx] - '0' ? 1 : 0));
31         ans %= MOD;
32     }
33
34     return ans;
35 }
36
37 int main() {
38     cin.tie(0) -> sync_with_stdio(false);
39
40     cin >> k >> d;
41     dp.resize(MAXK, vector<vector<int>>>(MAXD, vector<int>(2, -1)));
42
43     // conta no intervalo [0, k], mas o problema pede a resposta no intervalo [1, k]
44     // como 0 sempre será contado na resposta, basta subtrair 1.
45     cout << ((f(0, 0, 1) - 1) % MOD + MOD) % MOD << endl;
46 }
```

Fonte: o Autor

Código 5.10: Implementação da solução do problema *Bit Problem* em C++

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const int MAXN = 1 << 20; // tamanho máximo dos elementos
6 int n;
7
8 vector<int> superMask(vector<int> F) { // conta quantas super máscaras existem
9     for(int i = 0; i < 20; i++) // checa para cada posição de um conjunto
10         for(int mask = 1; mask < MAXN; mask++) // verifica para cada conjunto
11             if(((mask >> i) & 1) == 0) // caso não exista esse elemento no conjunto
12                 F[mask] += F[mask ^ (1 << i)]; // adiciona a possibilidade de estar
13     return F;
14 }
15
16 vector<int> subMask(vector<int> F) { // conta quantas sub máscaras existem
17     for(int i = 0; i < 20; i++)
18         for(int mask = 1; mask < MAXN; mask++)
19             if((mask >> i) & 1)
20                 F[mask] += F[mask ^ (1 << i)];
21     return F;
22 }
23
24 int getComp(int x) {
25     return (MAXN - 1) ^ x;
26 }
27
28 int main() {
29     cin.tie(0)->sync_with_stdio(false);
30
31     cin >> n;
32     vector<int> v(n);
33     for(auto& it: v) cin >> it;
34
35     vector<int> F(MAXN);
36     for(int i = 0; i < n; i++) // montagem do histograma
37         F[v[i]]++;
38
39     auto superM = superMask(F);
40     auto subM = subMask(F);
41
42     for(int i = 0; i < n; i++) {
43         cout << subM[v[i]] << ' ' << superM[v[i]] << ' ' << n - subM[getComp(v[i])] << '\n';
44     }
45 }
```

Fonte: o Autor

Capítulo 6

Considerações Finais

Neste trabalho foram introduzidos vários problemas que são resolvidos por programação dinâmica. As técnicas que aqui foram chamadas de clássicas, como o problema da mochila e do troco, foram apresentadas e também foi mostrado que, apesar de clássicas, é possível incrementar a dificuldade com uma variação dessas técnicas, como mostra o Capítulo 5. Como visto na Tabela 2.1, é necessário que o competidor tenha o conhecimento básico de técnicas de programação dinâmica, caso queira ter um bom desempenho, visto que é um conteúdo recorrente na Maratona de Programação.

A apresentação de técnicas mais avançadas, como o problema da soma de subconjuntos e a DP de dígitos, teve como objetivo apresentar novas técnicas para competidores mais experientes, que não costumam ser as primeiras técnicas avançadas, como as otimizações de programação dinâmica.

O trabalho tem um conteúdo educativo, com o propósito de preparar o competidor para a Maratona de Programação. Por isso, existe um capítulo exclusivamente para resolução de problemas, além de uma lista com sugestões de problemas que o competidor é incentivado a resolver.

6.1 Trabalhos Futuros

Existem muitas outras técnicas e truques que podem ser apresentados e desenvolvidos. Ainda no escopo de programação dinâmica, existem tópicos como:

- otimização de programação dinâmica – Divide and Conquer (D&C);
- otimização de programação dinâmica – Knuth;
- otimização de programação dinâmica – Convex Hull Trick (CHT);
- otimização de programação dinâmica com estrutura de dados;

- programação dinâmica de valor esperado;
- programação dinâmica de componentes conexos;
- programação dinâmica de intervalo; e
- otimização de programação dinâmica em árvores.

Outros trabalhos futuros podem envolver o estudo e a comparação entre implementações *bottom-up* e *top-down*, ou a relação de programação dinâmica com algoritmos de grafos, como o algoritmo de Dijkstra [19]. Uma última sugestão é aumentar a base de conhecimento em português sobre a maratona e realizar trabalhos em conteúdos distintos, como processamento de string, algoritmos gulosos, estrutura de dados, teoria dos números, álgebra, entre outros.

Referências

- [1] Li, Yujia, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu e Oriol Vinyals: *Competition-level code generation with alphacode*. Science, 378(6624):1092–1097, 2022. <https://www.science.org/doi/abs/10.1126/science.abq1158>. 1
- [2] Kostka, Bartosz: *Sports programming in practice*. Tese de Mestrado, University of Wrocław, 2021. 1, 6
- [3] Yuen, Kevin K. F., Dennis Y. W. Liu e Hong Va Leong: *Competitive programming in computational thinking and problem solving education*. Computer Applications in Engineering Education, 31(4):850–866, 2023. <https://onlinelibrary.wiley.com/doi/abs/10.1002/cae.22610>. 1
- [4] Bandeira, Ian Nery, Thiago Veras Machado, Vitor F. Dullens e Edna Dias Canedo: *Competitive programming: A teaching methodology analysis applied to first-year programming classes*. Em *2019 IEEE Frontiers in Education Conference (FIE)*, páginas 1–8, 2019. 1
- [5] ICPC: *Icpc fact sheet 2024*. <https://icpc.global/community/history/factsheet/world-finals-2024-factsheet.pdf>. Acessado: 10-02-2025. 1
- [6] SBC: *Regras da maratona de programação sbc*. <https://maratona.sbc.org.br/sobre/regras.html>. Acessado: 10-02-2025. 1
- [7] Halim, S., F. Halim e S. Effendy: *Competitive Programming 4 - Book 1: The Lower Bound of Programming Contests in the 2020s*. Número 1. 1 em *Competitive Programming 4: The New Lower Bound of Programming Contests in the 2020s*. Lulu.com, 2018, ISBN 9781716745522. <https://books.google.com.br/books?id=pOrMzQEACAAJ>. 2, 8
- [8] Laaksonen, Antti: *Competitive programmer's handbook*, 2018. Available online at <https://cses.fi/book>. 2, 8, 10, 22
- [9] Tommasini, Stefano: *Programação dinâmica*, 2014. Monografia de Trabalho de Conclusão de Curso, Universidade de São Paulo, N° USP: 7278080. 2, 3

- [10] Mattioli, Lucas Vasconcelos: *Comparativo entre a estratégia gulosa e a programação dinâmica para o problema do troco com sistemas de moedas canônicos*, 2018. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Software) — Universidade de Brasília, Brasília, 2018. 59 f., il. 3
- [11] Santos, Pedro Vítor Bortolli: *Ancestral comum mais próximo entre dois vértices de uma árvore*, novembro 2019. Monografia de Trabalho de Conclusão de Curso, Universidade de São Paulo. 3
- [12] Diks, K. *et al.*: *Looking for a challenge? the ultimate problem set from the university of warsaw programming competitions*. Relatório Técnico, University of Warsaw, 2012. 5
- [13] Skiena, Steven S. e Miguel A. Revilla: *Programming challenges: the programming contest training manual*. SIGACT News, 34(3):68–74, setembro 2003, ISSN 0163-5700. <https://doi.org/10.1145/945526.945539>. 6
- [14] Bloomfield, Aaron e Borja Sotomayor: *A programming contest strategy guide*. Em *Proceedings of the 47th ACM Technical Symposium on Computing Science Education, SIGCSE '16*, página 609–614, New York, NY, USA, 2016. Association for Computing Machinery, ISBN 9781450336857. <https://doi.org/10.1145/2839509.2844632>. 6
- [15] OBI: *Site da olimpíada brasileira de informática*. <https://olimpiada.ic.unicamp.br/>. Acessado: 12-02-2025. 7
- [16] SBC: *Regras da maratona de programação sbc*. https://maratona.sbc.org.br/primfase24/contest-pf2024/info_maratona.pdf. Acessado: 12-02-2025. 9
- [17] Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest e Clifford Stein: *Introduction to Algorithms*. MIT Press, 3rd edição, 2009, ISBN 978-0-262-03384-8. 9, 10, 12
- [18] Kirk, D.E.: *Optimal Control Theory: An Introduction*. Dover Books on Electrical Engineering Series. Dover Publications, 2004, ISBN 9780486434841. <https://books.google.com.br/books?id=fCh2SAtWIdwC>. 10
- [19] Sniedovich, Moshe: *Dijkstra's algorithm revisited: The dynamic programming connexion*. Control and Cybernetics, 35, janeiro 2006. 11, 22, 60
- [20] Van, C.L. e R.A. Dana: *Dynamic Programming in Economics*. Dynamic Modeling and Econometrics in Economics and Finance. Springer US, 2003, ISBN 9781402074097. <https://books.google.com.br/books?id=pwnSgxX0AVkC>. 11
- [21] Delisi, Charles: *Cooperative phenomena in homopolymers. an alternative formulation of the partition function*. Biopolymers, 13(7):1511–1512, 1974. <https://onlinelibrary.wiley.com/doi/abs/10.1002/bip.1974.360130719>. 11
- [22] Dreyfus, Stuart: *Richard bellman on the birth of dynamic programming*. Operations Research, 50(1):48–51, 2002. <https://doi.org/10.1287/opre.50.1.48.17791>. 11

- [23] Bellman, Richard: *Eye of the Hurricane*. WORLD SCIENTIFIC, 1984. <https://www.worldscientific.com/doi/abs/10.1142/0076>. 11
- [24] Mathews, G. B.: *On the partition of numbers*. Proceedings of the London Mathematical Society, s1-28(1):486–490, 1896. <https://londmathsoc.onlinelibrary.wiley.com/doi/abs/10.1112/plms/s1-28.1.486>. 12
- [25] Salkin, Harvey M e Cornelis A De Kluyver: *The knapsack problem: a survey*. Naval Research Logistics Quarterly, 22(1):127–144, 1975. 12
- [26] Kadane, Joseph B.: *Two kadane algorithms for the maximum sum subarray problem*. Algorithms, 16(11), 2023, ISSN 1999-4893. <https://www.mdpi.com/1999-4893/16/11/519>. 12
- [27] CP-Algorithms: *Longest increasing subsequence*. https://cp-algorithms.com/sequences/longest_increasing_subsequence.html. Acessado: 11-02-2025. 23
- [28] Bentley, Jon: *Programming pearls: algorithm design techniques*. Commun. ACM, 27(9):865–873, setembro 1984, ISSN 0001-0782. <https://doi.org/10.1145/358234.381162>. 26
- [29] USACO-Guide: *Longest increasing subsequence*. <https://usaco.guide/gold/lis?lang=cpp>. Acessado: 11-02-2025. 41
- [30] Dilworth, R. P.: *A decomposition theorem for partially ordered sets*. Annals of Mathematics, 51(1):161–166, 1950, ISSN 0003486X, 19398980. <http://www.jstor.org/stable/1969503>, acesso em 2025-02-12. 41